



Fakultät für Elektrotechnik und Informatik

Computational Health Informatics

Integration von neuronalen Netzen in eine Desktopanwendung zur medizinischen Bildverarbeitung

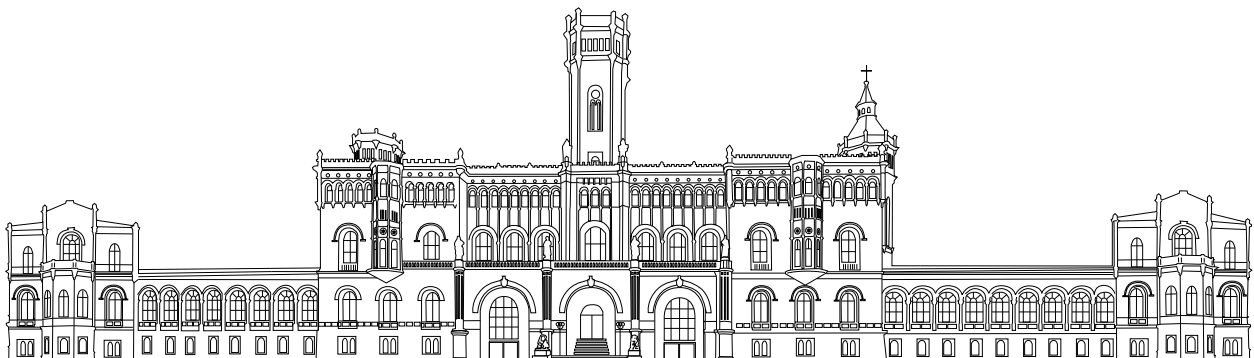
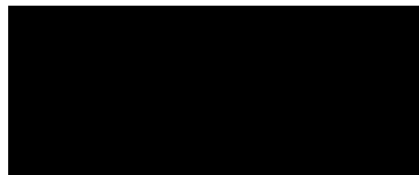
Masterarbeit im Studiengang Informatik(M.Sc.),
eingereicht von SVEN MICHAEL FECHNER am 18.04.2024

Erstprüferin:

Zweitprüfer:

Betreuer:

Matrikelnummer:



Abstract

The aim of this thesis is to find out to what extent neural networks can be integrated and used in desktop applications. Specifically, it is about analysing a video with a neural network and outputting the results of the analysis. The aim is to be able to use such an application in German medical practices. To this end, a market analysis was first carried out to find out which tools are currently used to write desktop applications. The resulting list, which takes into account the number of mentions, was then filtered in terms of the most suitable programming languages for *machine learning* and the integration of a neural network in *PyTorch* format. When analysing which operating systems are used in German medical practices, it became apparent that *Windows* is dominant here. The tools were therefore also filtered to determine whether they can write applications for *Windows*. It turned out that *PyTorch* automatically supports *C++*, *Python* and *Java*. In order to respect the time frame of this thesis, a total of three tools were selected, one for each programming language mentioned before. These are *UWP*, *PyQt6* and *JavaFX*. As a result, however, I was only able to write a fully functioning application with *PyQt6*. In *JavaFX* there were problems due to the *multithreading* and generally with the evaluation of the individual images. In *UWP* the required library *OpenCV* could not be integrated. However, the error here could also be due to the development environment recommended by *Microsoft*.

I was only able to collect performance results for the use of *PyQt6* and *JavaFX*. It turned out that the *PyQt6* application works both faster and more resource-efficiently than the *JavaFX* application. This leads to the assumption that *Python* generally seems to be better suited for such an endeavour than *Java*.

KAPITEL 1

Einleitung

1.1	Motivation	1
1.2	Ziel der Arbeit	2
1.3	Aufbau der Arbeit	2

1.1 Motivation

Mitte 2023 war Künstliche Intelligenz (KI) in aller Munde. Das lag hauptsächlich an der Veröffentlichung von ChatGPT. So war ChatGPT in der Lage Fragen von Nutzern zu analysieren und schnell und präzise zu antworten. Schnell haben Wissenschaftler begonnen vor KI zu warnen, sie könne in nicht allzu ferner Zukunft die Menschheit auslöschen. Es wurden Regularien und gar Verbote gefordert. [49, 50, 90, 144, 54, 59, 91, 48, 53]

Doch nicht nur KIs zur Beantwortung von Fragen existieren. Weniger bekannte KIs wie *Midjourney* oder *Leonardo Ai* sind in der Lage aus einem Eingabetext eindrucksvolle Bilder zu kreieren [96, 141].

Andere KIs wie *Stockfish* oder *Torch* sind in der Lage auf höchstem Niveau Schach zu spielen [132, 138]. Lässt man hingegen KIs wie *ChatGPT* Schach spielen, so bewegen die KIs ihre Bauern rückwärts, schlagen ihre eigenen Figuren, erschaffen Figuren aus dem Nichts und brechen generell eine Vielzahl an Regeln, was einen mehr erheitert als ängstlich werden lässt [66].

In der Medizin werden schon seit längerem Roboter eingesetzt. Diese sollen nicht nur den operierenden Ärzten das Arbeiten erleichtern und so für noch bessere OP-Ergebnisse sorgen, wie es mit dem *da Vinci* Roboter der Fall ist, sie übernehmen auch Therapie und Behandlungen sowie den Transport von medizinischen Materialien. [2]

Außerdem können Neuronale Netze, welches einen Teilgebiet der KI darstellt, in der Medizin genutzt werden. So können sie zum Beispiel bei der Tumorerkennung unterstützen und werden in diesem Teilgebiet immer besser [10, 86, 97, 92]. Studien haben gezeigt, dass Neuronale Netze für eine Tumorerkennung deutlich mehr Bilder verarbeiten können

mit einer sehr guten Präzision, als spezialisierte Ärzte in diesem Gebiet [65, 7, 29, 95]. An dem Punkt der Neuronalen Netze setzt diese Arbeit an und reiht sich so in ein Forschungsthema des Institut für Computational Health Informatics (CHI) an der Leibniz Universität Hannover (LUH) ein. Das CHI hat eine Kooperation mit einer Berliner Arztpraxis. In dieser Arztpraxis wird eine *Frenzelbrille* verwendet, eine spezielle Brille um das menschliche Auge aufzunehmen [56]. Innerhalb des Forschungsthemas wird eine Software entwickelt, welche mithilfe eines neuronalen Netzes die Pupille erkennt und verschiedene Eigenschaften wie Beschleunigung, Größe und Lichtreflex untersucht. Anhand dieser Ergebnisse sollen automatisiert Diagnosen gestellt werden können, ob zum Beispiel eine Gehirnerschütterung vorliegt.

1.2 Ziel der Arbeit

Wie eben genannt ist das Ziel dieser Arbeit eine Anwendung zu schreiben, welche mithilfe eines neuronalen Netzes das menschliche Auge erkennen und verschiedene Eigenschaften und Verhaltensweisen auswerten kann. Außerdem muss diese Anwendung auf in deutschen Arztpraxen gängigen Betriebssystemen lauffähig sein. Dabei soll die Anwendung allerdings nicht vollumfänglich sein. Viel mehr geht es hier um eine Art Machbarkeitsstudie. Dafür muss analysiert werden, mit welchen Werkzeugen aktuell Anwendungen entwickelt werden können. Ebenfalls muss erforscht werden, welche Betriebssysteme in deutschen Arztpraxen am verbreitetsten sind und welche Programmiersprachen sich allgemein am besten für die Implementierung eines neuronalen Netzes eignen. Anhand dieser Parameter werden einige Werkzeuge zur Anwendungsentwicklung ausgewählt. Mit diesen Werkzeugen wird dann eine vereinfachte Form einer Anwendung entwickelt, welche das Verwenden eines neuronalen Netzes zur Erkennung des menschlichen Auges implementiert und visuell darstellt. Außerdem sollen Zeiten gemessen werden, um herauszufinden, welche Anwendungen sich in der Praxis vermeintlich besser eignen würden.

Für diese Arbeit wurde jeweils ein neuronales Netz zur Detektion und zur Segmentierung bereitgestellt, die bereits darauf trainiert sind, das menschliche Auge zu erkennen. Ebenfalls wurde ein Video bereitgestellt, welches mithilfe der neuronalen Netze ausgewertet werden soll. Ein Beispiel für den Ablauf wurde in Form eines *Pythonskripts* ebenfalls dazugelegt. Im Idealfall kann die Anwendung aber nicht nur ein Video analysieren, sondern auch eine angeschlossene Kamera als Eingabequelle nutzen.

1.3 Aufbau der Arbeit

Diese Arbeit ist aus neun Kapiteln aufgebaut. In diesem ersten Kapitel wird neben der Motivation und dem Ziel der Arbeit und der Aufbau von dieser erläutert.

Im zweiten Kapitel wird auf die notwendigen Grundlagen eingegangen. Neben dem menschlichen Auge werden auch neuronale Netze, Anwendungsentwicklung sowie IT-Sicherheit bei Anwendungen im Gesundheitswesen erklärt.

Im dritten Kapitel wird kurz auf verwandte Arbeiten eingegangen, dessen Inhalte und Ergebnisse entweder in dieser Arbeit genutzt werden oder thematisch ähnlich sind.

Im vierten Kapitel wird die Methodik erläutert, mit welcher in dieser Arbeit vorgegangen wird. Dies betrifft sowohl die Vorgehensweise zur Auswahl der Werkzeuge, als auch die Vorgehensweise zur Entwicklung der Anwendungen.

Im fünften Kapitel wird anhand der Methodik recherchiert, welche Werkzeuge zur Entwicklung von Desktopanwendungen genutzt werden und sich am besten eignen würden. Anhand dieser Ergebnisse wird eine finale Auswahl getroffen.

Im sechsten Kapitel wird mithilfe der verschiedenen ausgewählten Werkzeuge jeweils eine vereinfachte Anwendung einer Desktopanwendung geschrieben und das neuronale Netz integriert. Zuvor wird die Einsteigerfreundlichkeit des jeweiligen Werkzeugs geprüft. Im Anschluss werden Performanztests mit dieser Anwendung durchgeführt.

Im siebten Kapitel werden die Ergebnisse des sechsten Kapitels miteinander verglichen.

Im achten Kapitel wird ein Fazit gezogen und ein Ausblick gegeben.

Methodik

4.1 Auswahl der Werkzeuge	31
4.2 Implementierung	32

In diesem Kapitel wird kurz die Vorgehensweise für diese Arbeit erläutert. Unterschieden wird dabei in zwei Abschnitte: Die Auswahl der Werkzeuge und die anschließende Implementierung.

4.1 Auswahl der Werkzeuge

Um die vermeintlich bestmöglichen Werkzeuge auszuwählen wird zuerst eine Internetrecherche betrieben um herauszufinden, mit welchen Werkzeugen aktuell Desktopanwendungen entwickelt werden. Um trotz subjektiver Blogeinträge einen möglichst objektiven Eindruck zu bekommen, wird eine zweistellige Anzahl an Blogeinträgen ausgewählt. Es werden alle genannten Werkzeuge festgehalten sowie gezählt, welches Werkzeug wie oft genannt wurde, um eine gewisse Gewichtung zu erzielen. Dabei gilt grundsätzlich: Je öfters ein Werkzeug genannt wurde, desto wahrscheinlicher ist es, dass es ein zielführendes Werkzeug für diese Arbeit sein könnte.

Nachdem eine (gewichtete) Liste aller möglichen Werkzeuge entstanden ist, muss diese Liste so gut wie möglich reduziert werden, damit das Ziel der Arbeit erreicht werden kann. Dafür wird zuerst danach gefiltert, welche Betriebssysteme die einzelnen Werkzeuge unterstützen. Es wird unterschieden zwischen den gängigen Desktopbetriebssystemen *Windows*, *Linux* und *macOS*. Außerdem wird *Cross Platform* aufgenommen, falls ein Werkzeug Anwendungen für mehrere Betriebssysteme bauen kann. Im Rahmen dieses Filters wird auch eine Recherche betrieben, welches Betriebssystem in deutschen Arztpraxen mutmaßlich das am weitesten verbreitete ist. Mit Hilfe dieses Ergebnisses wird die bisherige Liste also nicht nur neu sortiert, sondern auch potentiell erste Werkzeuge

aussortiert, sofern sie nicht das in deutschen Arztpraxen meistgenutzte Betriebssystem unterstützen.

Als nächstes wird geprüft, welche Programmiersprachen sich am Besten für neuronale Netze eignen. Wie schon für die Werkzeuge wird auch hier eine zweistellige Anzahl an Blogeinträgen rausgesucht für eine möglichst objektive Darstellung. Anschließend wird das Ergebnis dieser Liste verglichen mit dem neuronalen Netz und dessen unterstützten Programmiersprachen, welches für diese Arbeit bereits fertig zu Grunde liegt. Die Schnittmenge daraus stellt die möglichen Programmiersprachen da, welche für die Entwicklung einer Desktopanwendung im Rahmen dieser Arbeit in Frage kommen. Wichtig dabei ist, dass versucht wird, dass das neuronale Netz möglichst ohne Konvertierung zu implementieren. Sprich unterstützt das neuronale Netz *Java*, wird versucht eine *Java*-Anwendung zu entwickeln. Unterstützt das neuronale Netz auch *C*, wird dafür versucht, eine Anwendung in *C* zu schreiben. Es wird aber nicht eine Anwendung in *C* geschrieben, um ein Netz in *Java* zu verwenden.

Nun können die Werkzeuge, welche nach dem Filtern nach dem Betriebssystem noch übrig geblieben sind, den entsprechenden Programmiersprachen zugeordnet werden. Zusätzlich zu der eben entstandenen Schnittmenge wird es auch eine Kategorie *Andere* geben, die alle anderen Programmiersprachen für Desktopsysteme beinhaltet. Programmiersprachen für *Android*, *SmartTVs* und co. werden nicht gewertet.

Dank dieses Filters entsteht die finale Liste, aus welcher die Werkzeuge ausgewählt werden können. Dabei wird, sofern noch möglich, mindestens ein Werkzeug ausgewählt, mit welchem man Anwendungen speziell für das vorherrschende Betriebssystem entwickeln kann. Außerdem wird für jede Programmiersprache, welche in der Schnittmenge aus dem neuronalen Netz und den generell für neuronale Netze geeigneten Sprachen liegen, ein Werkzeug ausgewählt. Hier wird gegebenenfalls die Anzahl der Nennungen ein Entscheidungskriterium sein. Sollte aufgrund der bisher vorliegenden Informationen der gesamten Arbeit noch keine nachvollziehbare Entscheidung zwischen Werkzeugen gefällt werden können, werden die entsprechenden Werkzeuge miteinander verglichen, um so zu einer Entscheidung zu kommen.

4.2 Implementierung

Bei der Implementierung werden die ausgewählten Werkzeuge genutzt, um eine Desktopanwendung zu schreiben. Jedoch wird nicht direkt mit der Entwicklung der eigentlichen Anwendung begonnen. Vielmehr wird zuerst die Dokumentation durchgegangen. Dabei geht es weniger darum, wie umfangreich die Dokumentation ist, wie viele Kapitel existieren, und wie viel Text geschrieben wurde. Es geht eher darum herauszufinden, wie einsteigerfreundlich die Dokumentation ist. Dafür werden, sofern möglich, die Kapitel und Tutorials für den Einstieg in das Werkzeug durchgegangen. Wird erklärt, wie die Entwick-

lungsumgebung eingerichtet wird, wird dieser Anleitung gefolgt. Kann man selber etwas programmieren, wird dieses nachprogrammiert.

Nachdem der Einstieg in das Werkzeug geschafft ist, wird mit der Entwicklung der eigentlichen Desktopanwendung begonnen. Ziel dabei ist es, nicht einfach nur eine GUI zu bauen, über welche man das Video mit dem neuronalen Netz auswerten kann, sondern eine vereinfachte Version einer Anwendung, auf dessen Grundlage man eine Anwendung für eine Arztpraxis schreiben könnte. Bei dieser einfachen Anwendung geht es aber nur um die Optik, nicht um die Funktionalitäten. Unabhängig der Programmiersprachen werden die Anwendungen gleich aufgebaut sein. Die Optik der GUIs kann sich aufgrund der verschiedenen Werkzeuge aber unterscheiden.

Anschließend werden Performanztests durchgeführt. Es soll aber nicht nur das vorliegende Video in seiner Reinform untersucht werden. Das Video wird außerdem mit dem Videoeditor *VEGAS Pro 14 Steam Edition* bearbeitet. So soll sowohl der Einfluss der Videolänge, als auch die Anzahl an Bildern pro Sekunde auf die Performanz untersucht werden.

Außerdem gehört zu den Performanztests das Messen der Durchlaufzeiten der verschiedenen neuronalen Netze.

Falls während der Tests noch weitere interessante Aspekte auffallen, werden diese ebenfalls gemessen und ggf. rückwirkend schon abgeschlossenen Tests hinzugefügt.

Für die Tests werden zwei verschiedene Systeme genutzt. Einmal dem Entwicklungssystem, auf welchem die Anwendung entwickelt wurde. Und einmal auf einem Referenzsystem, auf welchem die im Anhang befindlichen Installationsanleitungen (siehe Anhang A) getestet wurden.

Die technische Ausstattung des Entwicklungssystems ist folgende:

- Typ: Desktop-PC
- Betriebssystem: Windows 11
- CPU: AMD Ryzen 5950X
- RAM: 64GB DDR4-3200
- GPU: NVIDIA GeForce RTX 2070 Super

Als Referenzsystem, welches auch von der Leistung her eher einem Desktop-PC einer Arztpraxis ähnelt und gleichzeitig eine Referenz für den mobilen Einsatz darstellt, wird folgendes verwendet:

- Typ: Microsoft Surface Pro 7
- Betriebssystem: Windows 11
- CPU: Intel i5-1035G4

- RAM: 8 GB DDR4-3733
- GPU: Intel Iris Plus Graphics 4GB

Alle Eindrücke und Ergebnisse werden in Kapitel 7 miteinander verglichen.

Werkzeuge zur Entwicklung von Desktopanwendungen

5.1	Marktanalyse	35
5.2	Filtern nach den Bedingungen	37
5.2.1	Filtern nach Betriebssystem	37
5.2.2	Filtern nach Integration von neuronalen Netzen	40
5.3	Finale Auswahl	41

In diesem Kapitel wird nach den bestmöglichen Werkzeugen gesucht, um eine Software zu bauen, welche die Aufgabenstellung bestmöglich erfüllt. Dafür werden zuerst durch eine Internetsuche verschiedene Werkzeuge gesammelt. Anschließend werden alle Werkzeuge herausgefiltert, welche die Aufgabenstellung nicht erfüllen können. Sollten danach noch mehr Werkzeuge übrig bleiben, als behandelt werden können, wird versucht Vor- und Nachteile der Werkzeuge aufzulisten. Basierend auf diesen Ergebnissen werden die bestmöglichen Werkzeuge ausgesucht und im späteren Verlauf dieser Arbeit verwendet.

5.1 Marktanalyse

Um herauszufinden, welche Werkzeuge zur Entwicklung von Desktopanwendungen heutzutage verwendet werden, wurde eine Internetrecherche mithilfe von Google und dem Suchstring *best software to create desktop application* durchgeführt. Es wurden keine weiteren Filter eingestellt. Bei den ausgewählten Blogbeiträgen wurde darauf geachtet, dass sie möglichst aktuell¹ sind. Außerdem wurden nur Blogbeiträge berücksichtigt, welche allgemeine Werkzeuge aufzählen. Das bedeutet, dass Blogbeiträge, die ausschließlich Werkzeuge zu einer einzigen Programmiersprache² nennen oder auf ein einzelnes

¹ Innerhalb der letzten fünf Jahre

² z.B. Python

Betriebssystem³ zugeschnitten sind, nicht berücksichtigt wurden. Daraus resultiert, dass zum Finden einer entsprechenden Anzahl an Einträgen für eine möglichst objektive Übersicht eine Top 50 Suche notwendig ist.

Die 14 gesammelten Blogeinträge liefern in ihrer Gesamtheit einen guten Überblick darüber, mit welchen Werkzeugen aktuell Desktopanwendungen entwickelt werden. Dabei wurden die Nennungen der Blogeinträge gezählt, und in Tabelle 5.1 zusammengestellt.

Tabelle 5.1: Auswertung der Nennungen von Werkzeugen zur Desktopanwendungsentwicklung in den verschiedenen Blogeinträgen

Werkzeug \ Quelle	[188]	[195]	[179]	[184]	[180]	[185]	[175]	[168]	[187]	[137]	[191]	[190]	[167]	[1]	Total
Tauri	x			x						x			x		4
Electron	x	x	x	x	x	x	x	x	x	x	x	x	x	x	14
Neutralino.js	x			x									x		3
Xojo	x	x		x	x					x			x		6
OS.js	x									x			x		3
WPF (Toolkit) ¹	x	x	x	x		x			x	x	x	x	x	x	11
8th Dev	x			x	x								x		4
Flutter for Desktop ²	x			x			x	x		x	x		x		7
Haxe	x			x	x								x		4
Enact	x			x						x			x		4
UWP	x	x	x	x		x			x	x	x	x	x	x	11
Xamarin.Forms	x			x									x		3
Winforms	x		x	x					x		x	x	x		7
Cocoa		x	x	x		x			x		x		x	x	8
Swing		x				x			x		x	x		x	6
JavaFX		x		x			x					x			4
Qt		x		x			x					x			4
Lazarus		x						x							2
NW.js				x	x										2
SwiftUI				x											1
GTK				x											1
B4J					x			x							2
Kivy					x			x							2
Enyo					x										1
WINDEV Express					x										1
FreePascal								x							1

¹ Da *WPF Toolkit* auf *WPF* basiert, wurden die Nennungen für beide Anwendungen zusammengefasst [162]

² Da eine Desktopanwendung entwickelt werden soll, wurden die Nennungen für *Flutter* und *Flutter for Desktop* zusammengefasst

In Tabelle 5.1 lässt sich erkennen, dass jeder Blogeintrag *Electron* empfiehlt. Aber auch *WPF* und *UWP* haben es mit 11/14 möglichen Nennungen zweistellig geschafft. Sortiert man nun die Tabelle nach der Anzahl der Nennungen ergibt sich das in Tabelle 5.2 zu sehende Bild.

³z.B. macOS

Tabelle 5.2: Ergebnis der Tabelle 5.1 sortiert nach Anzahl der Nennungen, anschließend alphabetisch

Werkzeug	#Nennungen
Electron	14
WPF (Toolkit) ¹	11
UWP	11
Cocoa	8
Flutter for Desktop ²	7
Winforms	7
Swing	6
Xojo	6
8th Dev	4
Enact	4
Haxe	4
JavaFX	4
Qt	4
Tauri	4
Neutralino.js	3
OS.js	3
Xamarin.Forms	3
B4J	2
Kivy	2
Lazarus	2
NW.js	2
Enyo	1
FreePascal	1
GTK	1
SwiftUI	1
WINDEV Express	1

¹ Da *WPF Toolkit* auf *WPF* basiert, wurden die Nennungen für beide Anwendungen zusammengefasst [162]

² Da eine Desktopanwendung entwickelt werden soll, wurden die Nennungen für *Flutter* und *Flutter for Desktop* zusammengefasst

5.2 Filtern nach den Bedingungen

Wie im Kapitel 4.1 beschrieben, muss das Werkzeug in der Lage sein, eine Desktopanwendung für ein in deutschen Arztpraxen gängiges Betriebssystem zu entwickeln und neuronale Netze, welche im *PyTorch* Format trainiert sind, integrieren zu können.

5.2.1 Filtern nach Betriebssystem

Zuerst wird geschaut, welche Betriebssysteme von den verschiedenen Werkzeugen unterstützt werden. Dabei vermute ich, dass nur die gängigen Betriebssysteme *Windows*, *Linux* und *macOS* für diese Arbeit relevant sein werden.

Um Tabelle 5.3 vernünftig auswerten zu können, muss noch ein Blick darauf geworfen werden, welches Betriebssystem in deutschen Arztpraxen vorrangig genutzt wird. Nach

Tabelle 5.3: Ergebnis der Tabelle 5.2 sortiert nach unterstützten Betriebssystemen der entwickelten Anwendung

Cross Platform	Windows	Linux	macOS
Electron[47]	WPF (Toolkit) ¹ [41]		Cocoa [33]
Flutter for Desktop ² [42]	UWP [156]		SwiftUI [133]
Swing [76]	Winforms [40]		
Xojo [164]	WINDEV Express [158]		
8th Dev [4]			
Enact [184]			
Haxe [68]			
JavaFX [195]			
Qt [126]			
Tauri [35]			
Neutralino.js [100]			
OS.js [114, 115]			
Xamarin.Forms [163]			
B4J [9]			
Kivy [89]			
Lazarus [5]			
NW.js [101]			
Enyo [180]			
FreePascal [55]			
GTK [67]			

¹ Da *WPF Toolkit* auf *WPF* basiert, wurden die Nennungen für beide Anwendungen zusammengefasst [162]

² Da eine Desktopanwendung entwickelt werden soll, wurden die Nennungen für *Flutter* und *Flutter for Desktop* zusammengefasst

Quelle [135], welche sich selbst auf die Kassenärztliche Bundesvereinigung (KBV) beziehen, haben die zehn meistgenutzten Praxissoftwares im Q3 2021 einen Marktanteil von 62,3%. Alle zehn unterstützen *Windows*. Zwei der zehn unterstützen neben *Windows* noch weitere Betriebssysteme. Rechnet man die zwei raus, haben die acht meistgenutzten *Windows*-Exklusiv Praxissoftwares einen Marktanteil von 50,3%.

Die zu diesem Zeitpunkt aktuellste Veröffentlichung zu den installierten Praxissoftwares von der KBV ist von Q4 2022 [73]. Vergleicht man die Zahlen aus [73] mit den Top 20 pro Fachgruppe [136], stellt man fest, dass die Zahlen von [73] alle Fachgruppen einschließen. Außerdem fällt in [136] auf, dass die Fachgruppe der Psychotherapeuten den mit Abstand größten Einfluss auf den Marktanteil hat.

In Tabelle 5.4 wird ein Blick auf die zehn meistinstallierten Praxissoftwares von Q4 2022 geworfen mit ihrem Gesamtmarktanteil. Zusätzlich wird geprüft, ob diese auch in den Top 20 der Fachgruppen Allgemeinmediziner und Augenärzte vorkommen (angegeben in Position in der jeweiligen Liste) und welchen Marktanteil sie an die jeweilige Top 20 haben nach den Quellen [73] und [136].

In Tabelle 5.4 sieht man, dass die insgesamt zehn meistinstallierten Praxissoftwares immer noch einen Marktanteil von 61,2% haben.

Tabelle 5.4: Die zehn meistinstallierten Praxissoftwares in Deutschland nach der KBV verglichen mit deren Installationen in den Fachgruppen Allgemeinmediziner und Augenärzte

Praxissoftware	Insgesamt		Allgemeinmediziner		Augenärzte	
	Position	Marktanteil ¹	Position	Marktanteil ¹	Position	Marktanteil ¹
MEDISTAR	1	9,6	2	11,6	1	20,7
Elefant	2	9,2	-	-	-	-
PSYPRACT	2	9,2	-	-	-	-
TURBOMED	4	7,4	1	13,9	5	10,3
x.isynet	5	6,1	3	9,4	7	3,8
Medical Office	6	4,3	5	6,9	8	3,3
Epikur	7	4,1	-	-	-	-
x.concept	8	4,0	4	7,8	-	-
ALBIS	9	3,7	6	6,8	13	1,1
SMARTY	10	3,6	-	-	-	-

¹ in %, gerundet auf eine Nachkommastelle

Von den zehn insgesamt meistinstallierten Praxissoftwares kommen sechs in den Top 20 der meistinstallierten Praxissoftwares bei den Allgemeinmediziner vor mit einem Top-20-Marktanteil von 56,4%.

In den Top 20 der meistinstallierten Praxissoftwares bei den Augenärzten kommen nur fünf der zehn insgesamt meistinstallierten Praxissoftwares vor. Diese fünf erreichen hier insgesamt einen Top-20-Marktanteil von 37,2%.

In der Liste von [135] sind alle sechs Praxissoftwares inkl. deren benötigten Betriebssysteme zu finden, die nach Tabelle 5.4 bei Allgemeinmediziner oder Augenärzten verwendet werden. Dabei kann man erkennen, dass fünf der sechs Praxissoftwares *Windows*-exklusiv sind, *TURBOMED* läuft als mobile Anwendung zusätzlich noch auf mind. *iOS 12*.

Bei den Augenärzten auf Platz zwei und Platz drei der meistinstallierten Praxissoftwares liegen *FIDUS* und *IFA-AUGENARZT* [136]. Diese sind exklusiv für Augenärzte [52, 71, 136]. *FIDUS* basiert auf dem *Microsoft Framework .net* [51]. Daher lässt sich sicher behaupten, dass *FIDUS* unter *Windows* funktioniert [98]. Ebenso läuft *IFA-AUGENARZT* unter *Windows* [113]. Zählt man deren Top-20-Marktanteile bei den Augenärzten hinzu⁴, dann erhält man einen Top-20-Marktanteil der Praxissoftwares von 70,6%.

Daraus lässt sich Schlussfolgern, dass die meisten Praxissoftwares *Windows*-exklusiv sind oder unter *Windows* laufen. Daher wird für den weiteren Verlauf der Arbeit die Aussage getroffen, dass *Windows* das meistgenutzte Betriebssystem für Praxissoftwares und somit in Arztpraxen darstellt.

Zurück zur Tabelle 5.3 lässt sich erkennen, dass in diesem Fall die Werkzeuge *Cocoa* und *SwiftUI* wegfallen, da sie nur auf *macOS* funktionieren. Alle anderen bilden die Menge der möglichen Werkzeuge, um eine der Aufgabenstellung entsprechenden Anwendung zu bauen.

⁴ *FIDUS* 17,8% und *IFA-AUGENARZT* 15,6%

5.2.2 Filtern nach Integration von neuronalen Netzen

Bevor dazu übergegangen wird, welche Werkzeuge benutzt werden können, um die Desktopanwendung zu bauen, wird zuvor geschaut, welche Programmiersprachen die vermeintlich besten sind, um mit neuronalen Netzen zu arbeiten. Auch hier wurde das Internet via Google wieder nach Blogeinträgen durchsucht, diesmal mit dem Suchstring *best programming language for neural networks*. Da es aber keine Blogeinträge gibt, die spezifisch auf neuronale Netze zugeschnitten sind, wird ein Blogeintrag, welcher ML als Allgemeinthema behandelt, als ausreichend angesehen. Außerdem wurde in diesem Fall nach Ergebnissen gefiltert, welche frühestens am 01.01.2022 erschienen sind oder nach dem 01.01.2022 zum letzten Mal aktualisiert wurden. Begründet liegt die Entscheidung in der Dynamik des Bereichs ML.

Tabelle 5.5: Auswertung der Nennungen von Programmiersprachen für Maschinelles Lernen in den verschiedenen Blogeinträgen

Sprache	[176]	[183]	[181]	[189]	[165]	[177]	[178]	[192]	[166]	[186]	[194]	Total
Python	x	x	x	x	x	x	x	x	x	x	x	11
R	x	x		x	x	x	x	x	x	x	x	10
C++	x	x	x	x	x	x	x	x	x	x	x	11
Java	x	x	x	x	x	x	x	x	x	x	x	11
JavaScript	x	x	x	x	x	x			x		x	8
Julia		x	x	x		x		x	x	x	x	8
LISP		x		x								2
Golang			x			x						2
Scala				x		x				x		3
C#						x						1
Haskell						x				x	x	3
Wolfram						x						1
MatLab						x						1
Lua								x				1
Rust										x		1
Prolog										x		1
Lisp										x		1

Ohne Tabelle 5.5 nach Anzahl der Nennungen zu sortieren, fällt bereits auf, dass sechs Sprachen anscheinend gut geeignet sind, sie im Bereich des MLs zu benutzen. Namentlich sind diese Sprachen: *Python*, *Java*, *C++*, *R*, *JavaScript* und *Julia*. Aufgrund des großen Abstandes zur nächst meistgenannten Nennung, werden alle Sprachen, welche weniger als acht Nennungen haben, an dieser Stelle bereits aussortiert.

Wie bereits erwähnt liegt das zu verwendende neuronale Netz im *PyTorch* Format vor. Daher lohnt es sich an dieser Stelle zu überprüfen, welche Sprachen von *PyTorch* unterstützt werden. Ein Blick in die Dokumentation verrät, dass neben *Python* auch *Java* und *C++* unterstützt werden [125]. Daher ist es sinnvoll, die in Kapitel 5.2.1 übrig gebliebenen Werkzeuge nach diesen drei Programmiersprachen zu filtern. Sollte dabei herauskommen, dass ein Werkzeug für verschiedene Plattformen verschiedene Programmiersprachen unterstützt oder gar benötigt, so werden nur die Programmiersprachen gewertet,

welche für *Windows* benötigt werden, sofern erkennbar.

Tabelle 5.6: Ergebnis der Tabelle 5.3 sortiert nach unterstützten Programmiersprachen

Werkzeug	Python	Java	C++	Andere
Windows				
WPF (Toolkit) ¹ [169]				x
UWP [156]			x	x
Winforms [140]				x
WINDEV Express [159]		x	x	x
Cross Platform				
Electron [151]				x
Flutter for Desktop ² [28]			³	x
Swing [76]		x		
Xojo [155]				x
8th Dev [4]				x
Enact [184]				x
Haxe [68]			x	x
JavaFX [61]		x		
Qt [127]	x		x	x
Tauri [153, 134]				x
Neutralino.js [100]				x
OS.js [114]				x
Xamarin.Forms [163]				x
B4J [9]				x
Kivy [87, 88]	x ⁴			
Lazarus [94]				x
NW.js [101]				x
Enyo [180]				x
FreePascal [55]				x
GTK [67]	x		x	x

¹ Da *WPF Toolkit* auf *WPF* basiert, wurden die Nennungen für beide Anwendungen zusammengefasst [162]

² Da eine Desktopanwendung entwickelt werden soll, wurden die Nennungen für *Flutter* und *Flutter for Desktop* zusammengefasst

³ Wenn man mit *Flutter* eine *Windows*-Anwendung schreibt, wird eine *runner app* in *C++* generiert. Da die aber hauptsächlich eine Art *Container* für die eigentliche Anwendung darstellt, wird hier *C++* nicht mit aufgenommen

⁴ *Kivy* benutzt zwar eine eigene Sprache, diese basiert aber auf *Python* und letztendlich werden *Python* Dateien erzeugt [87, 88]

Damit das im *PyTorch* Format vorliegende neuronale Netz möglichst ohne Konvertierung und den damit potentiell entstehenden Problemen integriert werden kann, werden an dieser Stelle alle Werkzeuge aussortiert, welche nicht *Python*, *Java* oder *C++* unterstützen. Dieses Kriterium sorgt für die in Abbildung 5.7 zu erkennende Tabelle.

5.3 Finale Auswahl

Die Tabelle 5.7 vereinigt die Ergebnisse aller vorherigen Filter, sodass hier nun eine finale Auswahl der genutzten Werkzeuge getroffen werden kann. Gemäß Kapitel 4 wird für

Tabelle 5.7: Ergebnis der Tabelle 5.6 zusammengefasst mit Anzahl der Nennungen nach Tabelle 5.1

Python	Java	C++	#Nennungen
Windows			
		UWP	11
	WINDEV Express	WINDEV Express	1
Cross Platform			
	Swing		6
		Haxe	4
	JavaFX		4
Qt		Qt	4
Kivy ¹			2
GTK		GTK	1

¹ Kivy benutzt zwar eine eigene Sprache, diese basiert aber auf *Python* und letztendlich werden *Python* Dateien erzeugt [87, 88]

jede Programmiersprache jeweils ein Werkzeug genutzt. Da nachweislich *Windows* das meistgenutzte Betriebssystem in deutschen Arztpraxen ist, wird folglich mindestens ein Werkzeug ausgewählt, welches speziell *Windows*-Anwendungen schreiben kann.

Wirft man einen Blick auf Tabelle 5.7, stellt man fest, dass nur noch zwei Werkzeuge übrig sind, welche speziell *Windows*-Anwendungen schreiben. *UWP* hat dabei elf Nennungen, *WINDEV Express* nur eine. Daher wird an dieser Stelle *UWP* als erstes Werkzeug ausgewählt. Da *UWP* *C++* als Programmiersprache unterstützt, fallen sämtliche andere Werkzeuge raus, welche exklusiv *C++* als Programmiersprache nutzen.

An dieser Stelle gilt es zu beachten, dass Microsoft *UWP* (mit *WinUI 2*) nicht weiterentwickelt und *UWP* kaum noch neue Features hinzugefügt werden. Allerdings bekommt *UWP* weiterhin Aktualisierungen, welche Bugs, die Zuverlässigkeit oder die Sicherheit betreffen, beheben. Der Nachfolger ist die *Windows App SDK* mit *WinUI3*. *WinUI3* soll dabei für ein moderneres Aussehen sorgen. Der Upgradeprozess von *UWP/WinUI 2* zu *Windows App SDK/WinUI 3* soll aber relativ einfach sein. [160, 6] Außerdem war in keinem der genannten Blogeinträge von der *Windows App SDK* die Rede, obwohl die überwiegende Mehrheit ihr Erscheinungs- oder letztes Aktualisierungsdatum weit nach Bekanntgabe von *Windows App SDK* hat.

Daher habe ich mich im Rahmen dieser Arbeit dazu entschieden den Blogeinträgen und den zusätzlichen Informationen zu folgen und *UWP* nicht gegen *Windows App SDK* auszutauschen.

Für *Python* wird *Qt* bzw. die *Python*-Version *PyQt* verwendet, da die Arbeit [174] gezeigt hat, dass *PyQt* für eine sehr ähnliche Aufgabe wie die für diese Arbeit funktioniert. Die Frage ist allerdings, ob *PyQt5* oder *PyQt6* benutzt werden soll. Nach Quelle [119] gibt es keine großen Unterschiede zwischen den beiden Versionen. Außerdem empfiehlt Quelle [119] für neue Projekte mit *PyQt6* zu starten, daher wird in dieser Arbeit auch *PyQt6* verwendet. Eine kurze Debatte, ob anstatt *PyQt6* *PySide6* verwendet werden sollte, wird in

Kapitel 6.3 geführt.

Swing und *JavaFX* sind native *Java* Anwendungsentwicklungswerkzeuge, daher wird an dieser Stelle *WINDEV Express* aussortiert. Auch dass *WINDEV Express* nur ein mal genannt wurde, hat Gewicht bei der Aussortierung.

Swing ist hauptsächlich dafür da, um eine GUI zu schreiben. *JavaFX* hingegen eignet sich besser zum Schreiben von Desktopanwendungen, wie es im Rahmen dieser Arbeit auch der Fall ist. *Swing* beinhaltet zwar bereits mehr Komponenten für die GUI-Entwicklung als *JavaFX*, diese sind aber standardisiert. Mit *JavaFX* ist man allerdings in der Lage GUIs zu schreiben, welche fortschrittlicher aussehen und sich auch so anfühlen. Ebenfalls ist *Swing* bereits zu Ende entwickelt, bei *JavaFX* kommen noch immer neue Komponenten hinzu. Auch ist das *MVC-Pattern* in *JavaFX* deutlich besser umgesetzt als bei *Swing*. [77]

Aufgrund dessen wird für die *Java*-Anwendung *JavaFX* verwendet.

Implementierung

6.1	Design der Anwendung	46
6.1.1	MEDISTAR	46
6.1.2	TURBOMED	47
6.1.3	Endgültiges Design	48
6.2	UWP	49
6.2.1	Dokumentation	49
6.2.2	Entwicklung der Anwendung	57
6.2.3	Auswertung der Performanz	65
6.3	Qt	66
6.3.1	Dokumentation	66
6.3.2	Entwicklung der Anwendung	86
6.3.3	Auswertung der Performanz	93
6.4	JavaFX	100
6.4.1	Dokumentation	101
6.4.2	Entwicklung der Anwendung	107
6.4.3	Auswertung der Performanz	114

In diesem Kapitel werden die Anwendungen nach dem im Kapitel 4.2 beschriebenen Vorgehen entwickelt. Die dafür verwendeten Werkzeuge wurden im Kapitel 5 erarbeitet. Bevor jedoch die Entwicklung starten kann, muss das grundlegende Design der Anwendungen festgehalten werden.

6.1 Design der Anwendung

Um ein entsprechendes Design der Anwendung zu erarbeiten, wird der Aufbau von *TURBOMED* sowie *MEDISTAR* analysiert, da diese unter den drei meistinstallierten Praxissoftwares bei Allgemeinmedizinern und Augenärzten liegen und parallel dazu zu den zehn meistinstallierten Praxissoftwares gehören (siehe Tabelle 5.4).

6.1.1 MEDISTAR

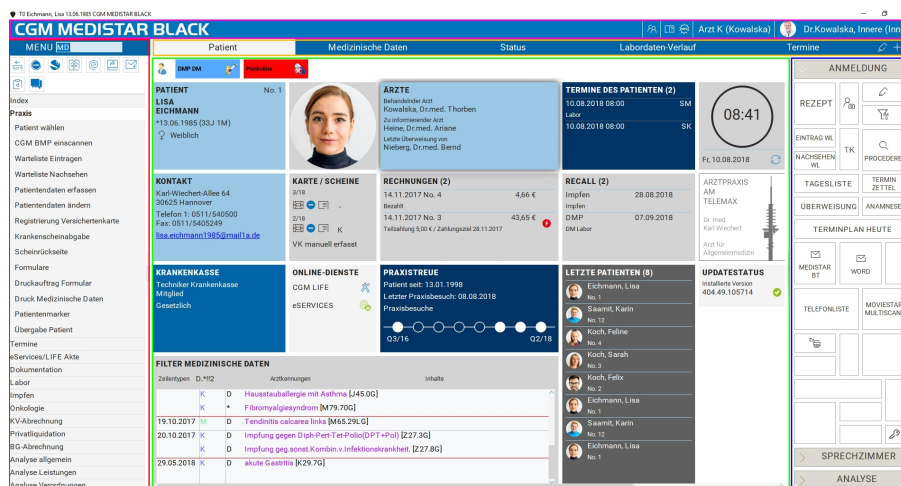


Abbildung 6.1: Analyse des Aufbaus der Praxissoftware *MEDISTAR* (Quelle: [15], verändert)

In Abbildung 6.1 ist ein Screenshot der Praxissoftware *MEDISTAR* zu sehen. Gleichzeitig ist dort der vermeintliche Aufbau und somit die Analyse des Designs farblich festgehalten.

- **Lila:** Ganz oben in der Anwendung ist der lila Kasten. In diesem Kasten steht der Name der Praxissoftware sowie der eingeloggte Nutzer. Außerdem befinden sich dort die Menüpunkte, womit der Nutzer seinen Account verwalten kann.
- **Rot:** Ganz links in der Anwendung ist der rote Kasten. In diesem Kasten befindet sich ganz oben eine Suchleiste und die Bezeichnung *Menu*. Darunter befinden sich Symbole zur Schnellnavigation. Wiederum darunter befindet sich das eigentliche Menü. Klickt man eine Kategorie an, klappen die Unterkategorien aus.
- **Orange:** Rechts von der Suchleiste innerhalb des roten Kastens befindet sich eine Art Tabnavigation. Sofern man in einen Bereich navigiert hat, werden hier im orangenen Kasten verschiedene Tabs, also Ansichten, angezeigt. Über die entsprechenden Flächen kann man die Ansichten wechseln. Der Bereich ist so hoch wie die Suchleiste und geht bis ganz rechts an den Rand der Anwendung.
- **Grün:** Rechts vom roten Kasten und unterhalb des orangenen Kastens befindet sich der grüne Kasten. In diesem Kasten werden alle Informationen des ausgewählten Bereichs und der Ansicht angezeigt.

- **Blau:** Rechts am Rand der Anwendung und rechts des grünen Kastens sowie unter dem orangenen Kasten befindet sich der blaue Kasten. Dieser Kasten beherbergt Elemente zur Weiterverarbeitung der im grünen Kasten zu sehenden Informationen. In diesem spezifischen Fall sind das Elemente zur Patientenverwaltung des im grünen Kasten zu sehenden Patienten.

6.1.2 TURBOMED

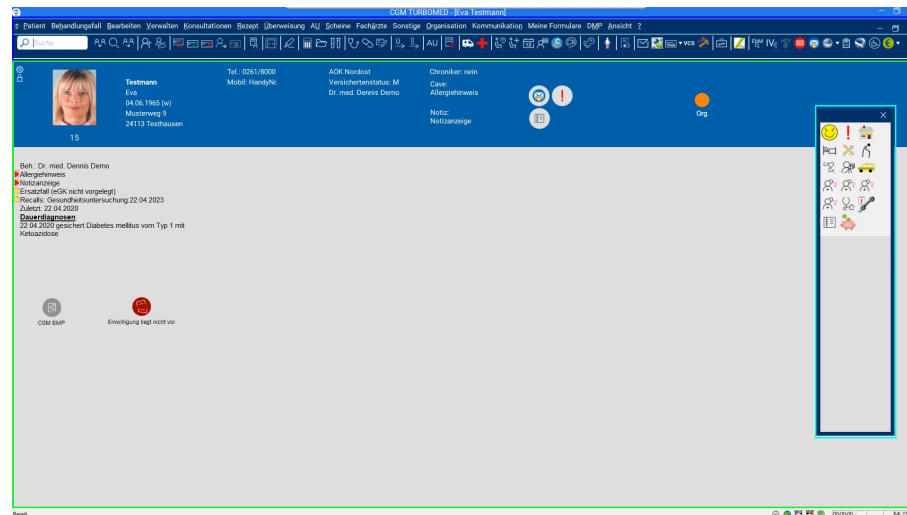


Abbildung 6.2: Analyse des Aufbaus der Praxissoftware *TURBOMED* (Quelle: [30], verändert)

In Abbildung 6.2 ist ein Screenshot der Praxissoftware *TURBOMED* zu sehen. Gleichzeitig ist dort der vermeintliche Aufbau und somit die Analyse des Designs farblich festgehalten.

Die Farben und deren Bedeutungen entsprechen der Erklärung aus Kapitel 6.1.1. Es fällt auf, dass in Abbildung 6.2 die Farben *Lila*, *Rot* und *Orange* fehlen, und die Farbe *Hellblau* neu hinzugekommen ist.

Hellblau: Hier rechts zu sehen über dem grünen Kasten liegend ist der hellblaue Kasten. In diesem Kasten sind Elemente enthalten, welche man in den grünen Kasten ziehen kann, um so seine Ansicht zu personalisieren [31].

Um den Kasten *Rot* zu finden, muss man sich das Hauptmenü von *TURBOMED* anschauen, welches analytisch in Abbildung 6.3 dargestellt ist.

In Abbildung 6.3 ist zu erkennen, dass das gesamte Fenster gemäß der Kategorisierung aus Kapitel 6.1.1 im roten Kasten liegt. Über diese modern wirkende Oberfläche findet die Navigation statt. Der Bereich *Lila* ist in dieses Hauptmenü integriert. Es wird zwar nicht erkenntlich, wie die Praxissoftware heißt und welcher Nutzer gerade eingeloggt ist, dennoch kommt man über das Hauptmenü in die Benutzerverwaltung.

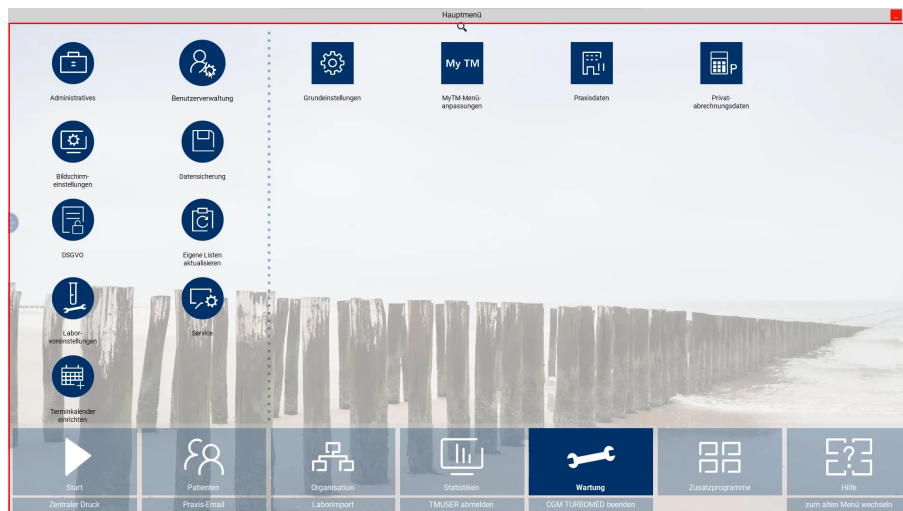


Abbildung 6.3: Analyse des Aufbaus des Hauptmenüs der Praxissoftware *TURBOMED* (Quelle: [30], verändert)

Der Bereich *Orange* ist in beiden Abbildungen nicht zu erkennen. Durch den Bereich *Hellblau* ist der Bereich *Orange* aber auch nicht notwendig.

6.1.3 Endgültiges Design

Was beide Praxissoftwares gemeinsam haben, ist, dass der Bereich *Grün* groß und mittig platziert ist. Ebenfalls ist der Bereich *Blau* in beiden Fällen zu erkennen, allerdings einmal oberhalb des grünen Bereiches und einmal rechts vom grünen Bereich. Die Bereiche *Rot* und *Lila* wurden im Fall von *TURBOMED* ausgelagert in eine eigene Oberfläche. Der Bereich *Orange* ist nur in einem Fall vorhanden, wird aber im anderen Fall durch den Bereich *Hellblau* ersetzt.

Die Anwendung selbst soll keinen komplizierten Aufbau erhalten. Daher wird auf eine extra Oberfläche verzichtet und das Menü so positioniert, wie es bei *MEDISTAR* der Fall ist. Allerdings, um einen ähnlich großen grünen Bereich wie bei *TURBOMED* zu ermöglichen, wird versucht dieses Menü ein- und ausklappbar zu gestalten. Andernfalls wird das Menü so groß wie nötig, aber so klein wie möglich, gehalten. Somit wird in der Anwendung der Bereich *Grün* groß und mittig platziert sein. Beide Softwares haben ein Menü rechts vom grünen Bereich. In einem Fall ist das zur Patientenverwaltung, im anderen Fall zur Personalisierung des grünen Bereiches. Für die hier zu entwickelnde Anwendung resultiert daraus, dass rechts des grünen Bereiches ebenfalls ein Bereich liegen wird, in welchem eine Art Verwaltung passend zum grünen Bereich liegen wird. Außerdem wird ein Bereich *Lila* den hier zu entwickelnden Anwendungen hinzugefügt aus Gründen der Vollständigkeit. Dieser wird wie bei *MEDISTAR* oberhalb des grünen, roten und blauen Bereiches liegen. Der beschriebene Aufbau ist nochmal in Abbildung 6.4 dargestellt.

Wichtig dabei ist, dass fast alle Flächen keine expliziten Funktionen haben werden, da nur eine vereinfachte Anwendung entwickelt wird, welche Bilddaten verarbeiten soll.

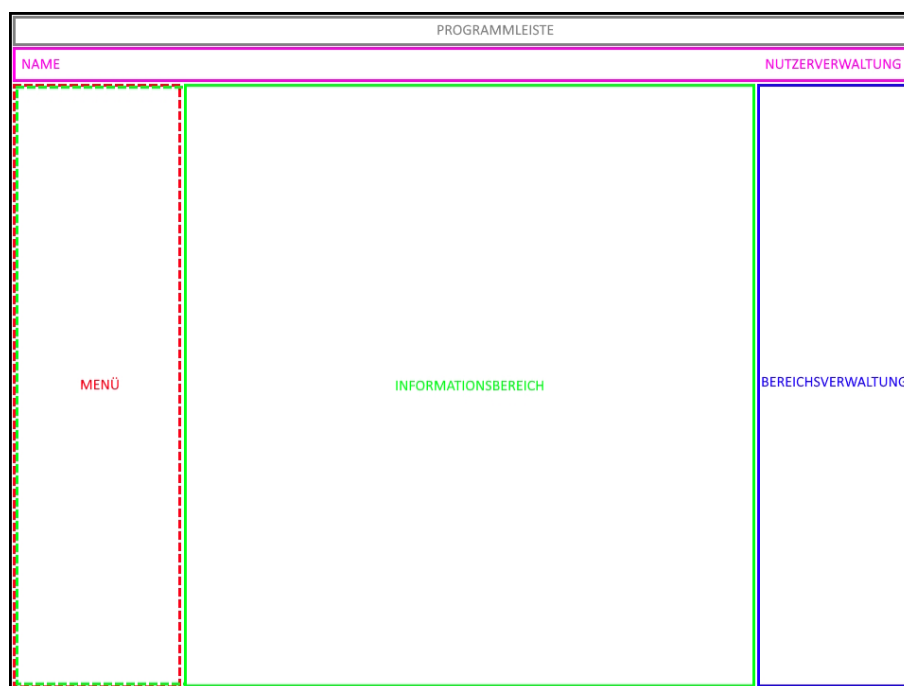
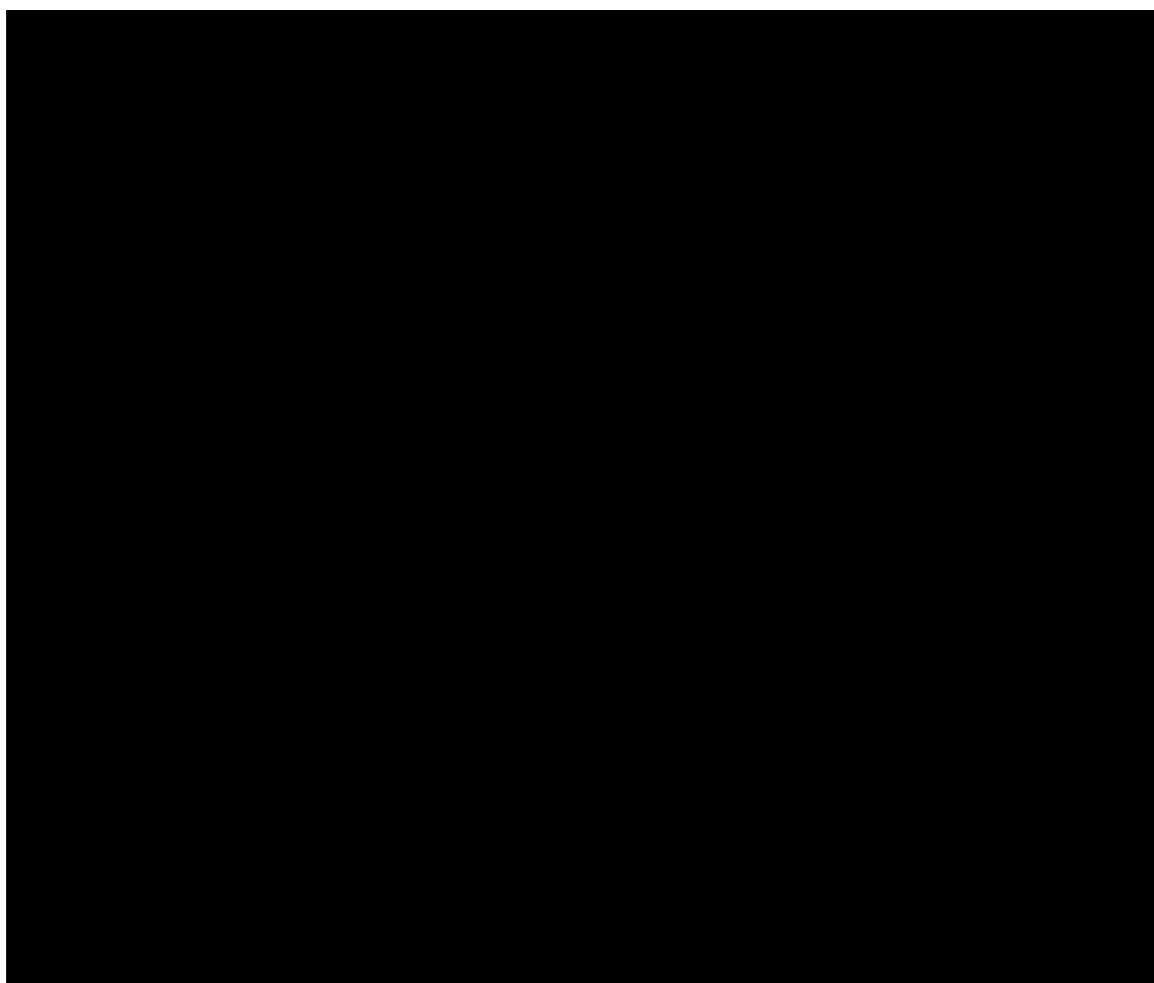
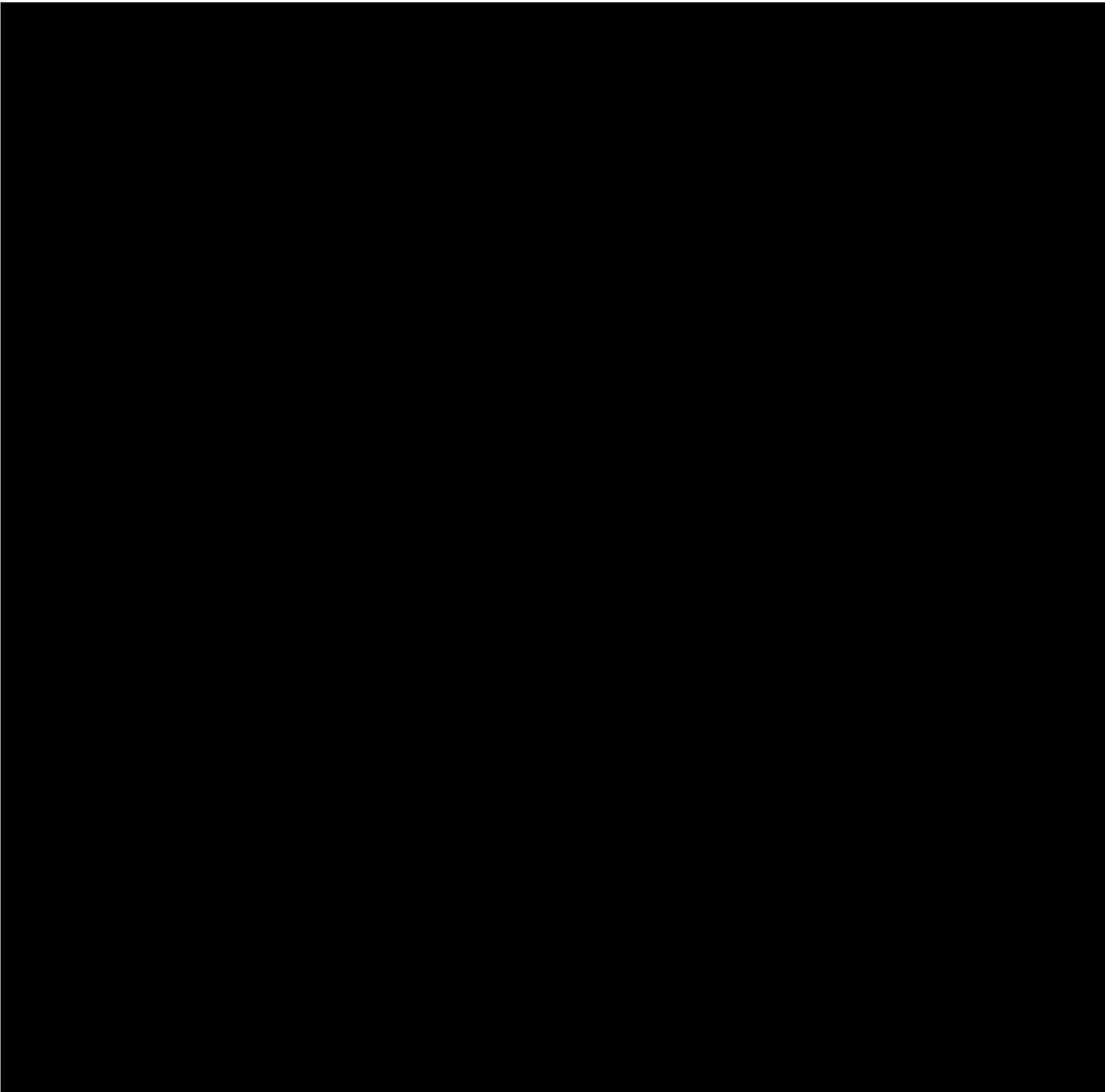


Abbildung 6.4: Aufbau der vereinfachten Anwendung

6.2 UWP





6.3.2 Entwicklung der Anwendung

Nach Bearbeiten der Tutorials kann mit dem Entwickeln der eigentlichen Anwendung begonnen werden. Aufgrund der zeitlichen Vorgaben für diese Abschlussarbeit wird sich dabei aber nur auf die Implementierung des neuronalen Netzes fokussiert und nicht vier verschiedene Seiten gebaut. Zum Bau der Anwendung benutze ich den *Qt Designer*, der bereits aus den Tutorials bekannt ist.

Auch wenn es *best practice* erst alle benötigten Flächen auf der Anwendung zu platzieren und diese dann erst anzuordnen, habe ich in einem ersten Schritt innerhalb weniger Minuten alle *Widgets* für den Hintergrund⁵⁷ hinzugefügt, wie in Abbildung 6.30 zu sehen. Wichtig ist, dass die minimale Fenstergröße der Anwendung auf eine Auflösung von 800x600 festgelegt ist, da diese Auflösung gerade so genug Platz für alle *Widgets* bietet. Jedoch wird die Anwendung standardmäßig mit einer Auflösung von 1024x768 geöffnet. Dadurch sollte auf modernen Bildschirmen sichergestellt sein, dass die Anwendung nicht direkt das gesamte Fenster einnimmt. Das *Menü-Widget* und das

⁵⁷Von der Funktion her hier vergleichbar mit den *RelativePanels* aus *UWP*

Bereichsverwaltung-Widget haben dabei eine festgelegte Breite von 150px⁵⁸, wie schon in der *UWP*-Anwendung. Das *Nutzerverwaltungs-Widget* hat eine festgelegte Höhe von 64px⁵⁹, ebenfalls wie in der *UWP*-Anwendung. Das *Informationsbereich-Widget* hat keine festgelegte Größe. Das *Widget* für die Graphen ist auf eine Höhe von 200px⁶⁰ festgelegt. Dadurch sollten sowohl die Graphen gut sichtbar sein, als auch im *Informationsbereich-Widget* genügend Platz übrig bleiben, um das ausgewertete Video anzuzeigen.

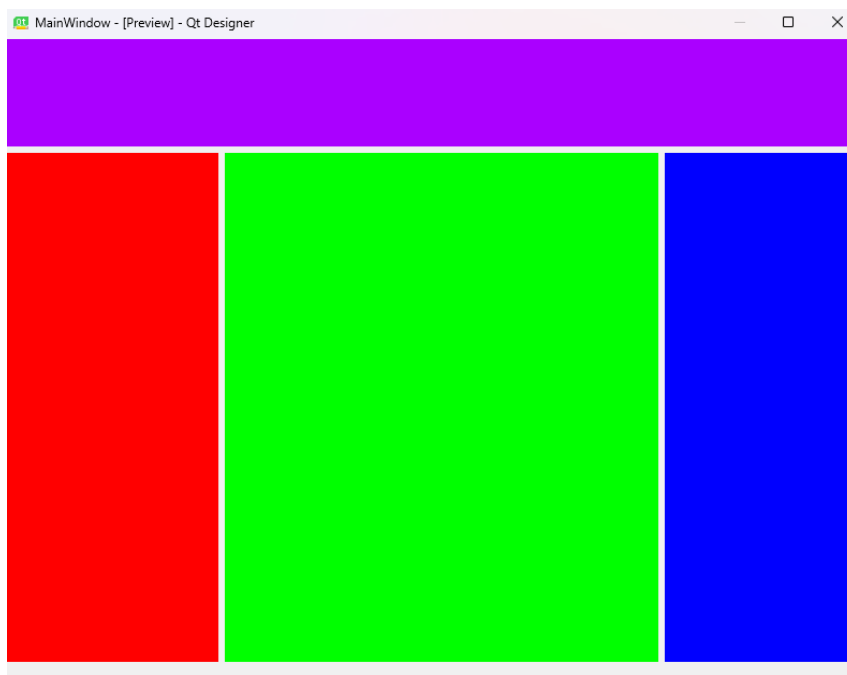


Abbildung 6.30: Das grundlegende Layout der *PyQt6* Anwendung

Um die Graphen getrennt vom Video behandeln zu können, habe ich mich dazu entschieden, noch ein weiteres *Widget* extra dafür hinzuzufügen, was in Abbildung 6.31 in einem dunkleren Grünton zu erkennen ist.

Nachfolgend habe ich erstmal der Bereichsverwaltung ein paar Texte und Buttons hinzugefügt. Diese beinhalten Informationen über die Anwendung des neuronalen Netzes. Dabei öffnet der Button *Netz laden* einen Dialog, wo man sich sein neuronales Netz aussuchen und in die Anwendung laden kann. Der Button *Video laden* tut im Prinzip das Gleiche, nur dass man hier das Video auswählt. Und der Button *Video auswerten* wertet das Video aus. Die dazugehörigen Text-Labels zeigen entweder die Bedienungsanleitung an, oder welches neuronale Netz bzw. Video geladen ist und wie lange die Auswertung der Videos gedauert hat. Diese Oberfläche ist in Abbildung 6.32 zu erkennen.

Da ich nun zwischen zwei verschiedenen Seiten navigieren wollte, habe ich versucht diese einzubinden. Dabei ist mir aufgefallen, dass aktuell beide Seiten als neues Fenster geöffnet werden. Der Grund dafür ist, dass ich im *Qt Designer* beide Seiten als *Window* erstellt habe. Was ich stattdessen hätte tun müssen, ist diese Seiten als *Widget* zu erstellen. Nur die Startseite muss ich als *Window* erstellen. In der Theorie benötigt

⁵⁸Festgelegt durch minimale und maximale Breite

⁵⁹Festgelegt durch minimale und maximale Höhe

⁶⁰Festgelegt durch minimale und maximale Höhe



Abbildung 6.31: Das grundlegende Layout der *PyQt6* Anwendung für die Seite des neuronalen Netzes

die Startseite nur ein *QStackedLayout*. Die Navigation kann man dann implementieren, wie man möchte. Mir hat dies allerdings die Möglichkeit aufgezeigt, das *Menü* und die *Nuterverwaltung* auf der Startseite zu implementieren und so aus den *Widgets* zu den verschiedenen Seiten entfernen zu können. Dadurch erspare ich mir die Mehrfachimplementierung der dazugehörigen Funktionen, wie ich es nach meinem Kenntnisstand bei *UWP* hätte tun müssen. Die *Widgets* zu den verschiedenen Seiten bestehen dann nur noch aus der *Bereichsverwaltung* und dem *Informationsbereich*. Damit die Startseite richtig angezeigt wird, also Texte nicht abgeschnitten werden, habe ich mich an dieser Stelle dazu entschieden, die Mindestgröße der Anwendung auf eine Auflösung von 1024x768 zu erhöhen. Nach diesen Änderungen sind die Anwendung aus wie in Abbildung 6.33 und Abbildung 6.34. Die rote Farbe für das *Menü* hatte ich nach der Umstrukturierung nicht wieder eingebaut. Dennoch sollte der Bereich eindeutig identifizierbar sein.

Bis auf dem Navigieren zwischen den Seiten, kann die Anwendung allerdings noch nicht viel. Alles für die Navigation notwendige ist in Listing 6.5 dargestellt, hier am Beispiel der Navigation zur Startseite. Zuerst muss die Startseite erzeugt werden. Daraufhin wird das erzeugte *Widget* dem *Stack* hinzugefügt. Als nächstes wird der Button, der für das Navigieren zur Startseite zuständig ist, mit der entsprechenden Navigationsmethode verbunden. Die Navigationsmethode selbst ändert nur, welcher Eintrag des *Stacks* angezeigt werden soll. In diesem Fall habe ich mich dazu entschieden vom *Stack* den Index des gewünschten Objekts zu lesen und das Ergebnis der Methode zum Wechseln zu übergeben, anstatt direkt einem *int* Wert. Dadurch navigiert die Methode noch immer zum gleichen *Widget*, selbst wenn sich die Reihenfolge der *Widgets* beim Bauen des *Stacks* ändern sollte. Andere Seiten werden analog zu diesem Vorgehen angebunden.

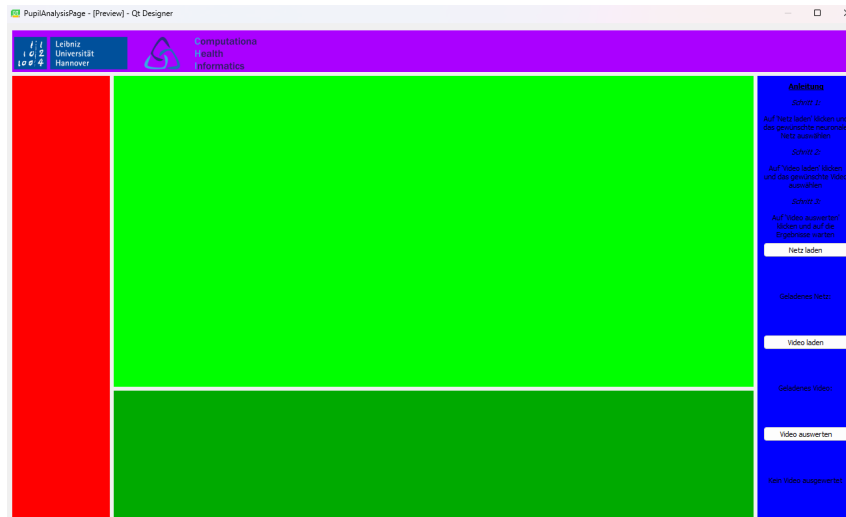


Abbildung 6.32: Das grundlegende Layout der *PyQt6* Anwendung für die Seite des neuronalen Netzes mit ersten Implementierungen



Abbildung 6.33: Die mit *PyQt6* gebaute Anwendung in einem ersten Entwurf. Es wird die Startseite angezeigt

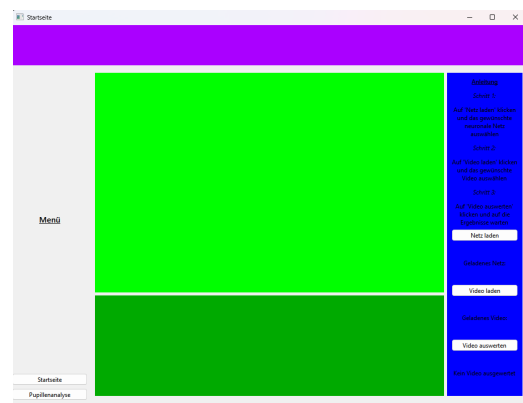


Abbildung 6.34: Die mit *PyQt6* gebaute Anwendung in einem ersten Entwurf. Es wird die *PupilAnalysisPage* angezeigt

```
self.startPage = StartPage()
self.stackedWidget.addWidget(self.startPage)

self.navToDefault_button.clicked.connect(self.navigateToDefault)

def navigateToDefault(self):
    self.stackedWidget.setCurrentIndex(self.stackedWidget.indexOf(self.startPage))
```

Listing 6.5: Der Quellcode zur Navigation in *PyQt6*

Nach dem Navigieren habe ich als nächstes die Buttons *Netz laden* und *Video laden* mit jeweils einer Funktion verbunden, um die entsprechende Datei mithilfe des Dateibrowsers herauszusuchen. Anschließend wird der unter dem entsprechenden Button liegende

Text aktualisiert mit der Information, welches neuronale Netz oder welches Video geladen wurde, und nochmals zentralisiert.

Als nächstes habe ich mich darum gekümmert, dass die Kernfunktionalität implementiert wird, das Analysieren eines Videos mithilfe eines neuronalen Netzes. Wie schon in *UWP* wird hierfür *OpenCV*⁶¹, verwendet, um die einzelnen Bilder des Videos zu extrahieren. Außerdem kommt *Ultralytics*⁶² hinzu, um die Bilder mit dem neuronalen Netz analysieren zu können. Zu guter Letzt wird auch *onnxruntime*⁶³ benötigt, da wir hier mit einem neuronalen Netz im *onnx* Format hantieren⁶⁴.

Der zur Anwendung des neuronalen Netzes auf eine Videoeingabe genutzte Quellcode ist in Listing 6.6 zu sehen. Bis auf die *Imports* wird der Quellcode aus Gründen der Lesbarkeit als Pseudocode dargestellt.

Der Code in Listing 6.6 beginnt mit einer Reihe an Vorbereitungen. Dabei wird dem Nutzer signalisiert, dass die Analyse gestartet wurde und das Programm in der Zeit einfrieren könnte. Außerdem wird eine 'grüne Wiese' aufgebaut, auf welcher die Analyse stattfinden wird. Danach werden alle relevanten Bibliotheken importiert. Da diese nur innerhalb dieser Methode genutzt werden, ist es ausreichend sie lokal zu importieren und nicht auf globaler Ebene. Da nach dem *Import* die eigentliche Analyse startet, wird an dieser Stelle erst noch die aktuelle Uhrzeit festgehalten. Danach wird das neuronale Netz als YOLO-Modell geladen. Dafür wird aus der Klassenvariable der Pfad gezogen. Anschließend wird überprüft, ob der Nutzer ein Video ausgewählt hat. Sollte kein Video ausgewählt sein, wird *OpenCV* die Kamera verwenden, ansonsten den Pfad des Videos. Nach einer Prüfung, ob die Kamera oder das Video geöffnet werden konnte, startet das Lesen des Videos, sofern es gelesen werden kann. Dabei wird Bild für Bild gelesen. Auf jedes gelesene Bild wird das YOLO-Modell angewendet. Abhängig davon, ob Detektion oder Segmentierung angewendet werden soll, wird ein anderer Codeabschnitt angewendet. In beiden Codeabschnitten wird aber die Mitte der erkannten Pupille festgehalten und die Bewegungen auf der X- und auf der Y-Achse im Vergleich zum Bild davor festgehalten und in dem entsprechenden Graphen angezeigt. Zusätzlich wird auf der Umfang der erkannten Pupille und die Größenveränderung im Vergleich zum Bild davor berechnet und im entsprechenden Graphen angezeigt. Außerdem wird jeweils die erkannte Pupille auf das Bild eingezeichnet. Dieses Bild wird dann umgewandelt, um dieses in der Anwendung anzeigen zu können. Dies geschieht solange wie neue Bilder gelesen werden können. Danach wird die Endzeit festgehalten, die Dauer berechnet und im entsprechenden *Label* angezeigt.

```
def analyze(self):  
    Text des entsprechenden Labels aktualisieren, dass die  
        Analyse gestartet wurde  
    Erstellen von leeren Plots  
    Leeren der PlotWidgets, um ueberlappende Graphen zu
```

⁶¹Heruntergeladen als Version: 4.9.0.80

⁶²Heruntergeladen als Version: 8.1.24

⁶³Heruntergeladen als Version: 1-17-1

⁶⁴Resultat aus der Anwendungsentwicklung mit *UWP*

vermeiden

Verbinden der leeren Plots mit den PlotWidgets

Initialisieren von Helfervariablen zur Berechnung der
Plotwerte

```
import cv2
from ultralytics import YOLO
from datetime import datetime
from math import pi as PI
import numpy as np
```

Festhalten der Startzeit der Analyse

Laden des neuronalen Netzes als YOLO Modell

Wurde kein Video geladen:

 Nutze die Kamera

Ansonsten:

 Nutze das Video

Ueberpruefen, ob die Kamera oder das Video geoeffnet werden
konnte. Falls nicht, wird eine Nachricht ins
entsprechende Label geschrieben

Solange wie das Video gelesen werden kann:

 Versuchen das naechste Bild des Videos zu lesen

 Wenn das Bild gelesen werden konnte:

 Anwenden des YOLO Modells auf das Bild

 Fuer jedes durch YOLO gelaufene Bild:

 Bei Segmentierung:

 Wenn eine Maske erkannt wurde:

 Auswaehlen der Maske und Anpassen an das
 OpenCV Format

 Fuer jeden Umriss:

 Ellipse erstellen

 X-/Y-Koordinaten sowie die Groesse
 des Umfangs ermitteln, die

 Veraenderung berechnen und **in** die
 entsprechenden Graphen schreiben

 Ellipse **in** das Bild malen

Ansonsten (Detektion):

Wenn ein Objekt vorhanden:

Box und Zentrum extrahieren

X-/Y-Koordinaten ermitteln, die

Veraenderung berechnen und **in** die
entsprechenden Graphen schreiben

Rechtecke erstellen, Ellipse berechnen
und Ellipse **in** das Bild malen

Groesse des Umfangs ermitteln, die
Veraenderung berechnen und **in** den
entsprechenden Graphen schreiben

Format des aktuellen Bildes bekommen und **in** ein
QImage umwandeln

QImage zu einer QPixmap machen und die QPixmap dem
Label zur Anzeige der Bilder hinzufuegen

Endzeit festhalten, Differenz zur Startzeit berechnen und
die Dauer im entsprechenden Label ausgeben

Listing 6.6: Der Quellcode als Pseudocode für die Analyse einer Videoeingabe mit einem neuronalen Netz in *Python*

Beim Testen der Analyse ist mir aufgefallen, dass das *ONNX* Netz zur Segmentierung keine Ergebnisse liefert. Das Ursprungsnetz, also das neuronale Netz zur Segmentierung, bevor es nach *ONNX* konvertiert wurde, funktioniert hingegen. Der Grund dafür ist mir nicht bekannt und aufgrund der zeitlichen Beschränkung dieser Arbeit kann ich dem auch nicht nachgehen.

Dann entstand noch die Problematik, dass wenn einmal ein Video geladen wurde, man nicht mehr über die Anwendung auf die Kamera zugreifen konnte. Daher habe ich der Anwendung einen Button zum Zurücksetzen des geladenen Videos und der Vollständigkeit halber ein Button zum Zurücksetzen des neuronalen Netzes hinzugefügt, welche nichts anderes tun, als die entsprechende Variable wieder auf *None* zu setzen. Außerdem fehlte noch eine Entscheidungsmöglichkeit, ob Segmentierung der Detektion angewendet werden soll. Ein *fuzzy*-Selektor, welcher den Pfad auf ein Schlüsselwort filtert, war mir dabei zu ungenau. Daher habe ich letztlich eine *Checkbox* hinzugefügt, welche die Entscheidung beinhaltet.

Zuletzt stand nur noch das Problem, dass die Bilder im *Nutzerverwaltungs-Widget* nur im *Qt Designer* angezeigt wurden, aber nicht in der ausgeführten Anwendung, wie in Abbildung 6.33 zu erkennen. Um dieses Problem zu beheben musste ich die *resource*-Datei, wie ich sie im *Qt Designer* erstellt habe, mit dem Konsolenbefehl `pyside6-rcc -o {Zieldateiname}.py {Quelldateiname}.qrc` in eine *Python*-Datei umwandeln und in dieser Datei noch den Import von *PySide6* nach *PyQt6* ändern. Danach wurden mir auch in der ausgeführten Anwendung die Bilder angezeigt. Nachdem ich noch die Farben entspre-

chend der mit *UWP* erstellten Anwendung angepasst habe, ist die Anwendung fertig. Die Startseite, welche man direkt nach dem Öffnen der Anwendung sieht, sieht aus wie in Abbildung 6.35. Die Seite für die Pupillenanalyse sieht aus wie in Abbildung 6.36, bevor das erste Video analysiert wurde, während einer Analyse sieht sie aus wie in Abbildung 6.37, und nach der Analyse wie in Abbildung 6.38.

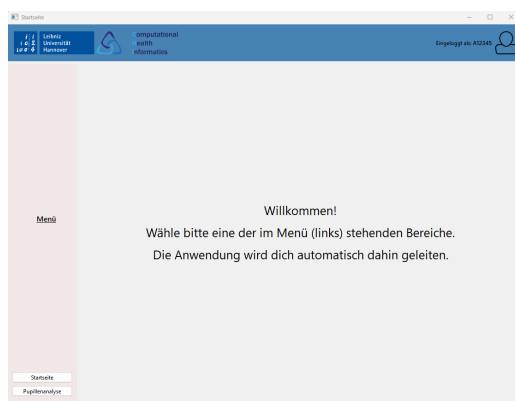


Abbildung 6.35: Die mit *PyQt6* entwickelte, fertige Anwendung auf der Startseite

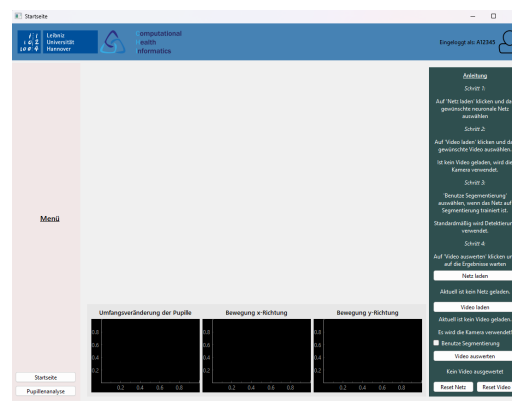


Abbildung 6.36: Die mit *PyQt6* entwickelte, fertige Anwendung auf der leeren Pupillenanalyseseite

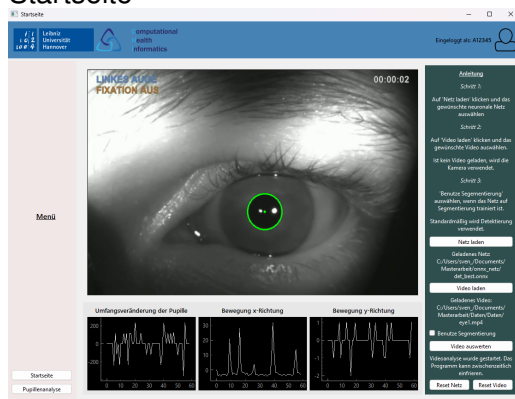


Abbildung 6.37: Die mit *PyQt6* entwickelte, fertige Anwendung auf der Pupillenanalyseseite während der Analyse

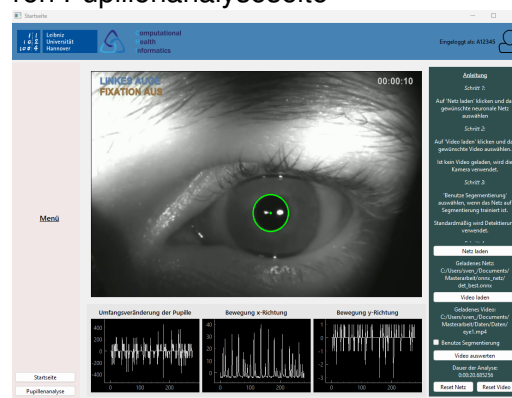


Abbildung 6.38: Die mit *PyQt6* entwickelte, fertige Anwendung auf der Pupillenanalyseseite nach der Analyse

6.3.3 Auswertung der Performanz

Entwicklungssystem

Bevor die Performanz ausgewertet wird, also die Dauer der Verarbeitung des Videos mit einem neuronalen Netz, wird das Originalvideo erst mithilfe von *Vegas Pro 14 Steam Edition* verändert. Das Original hat eine Dauer von knapp mehr als zehn Sekunden und 26.66 Bilder pro Sekunde. Eine erste Version wird auf 30 Bilder pro Sekunde gesetzt⁶⁵ und die Dauer entsprechend angepasst. Diese ist bei knapp über neun Sekunden. Zusätzlich wird noch eine Version mit 60 Bilder pro Sekunde⁶⁶ und entsprechender Dauer,

⁶⁵Entspricht einen Umrechnungsfaktor von 1,125

⁶⁶Entspricht einen Umrechnungsfaktor von 2,251

sowie 15 Bilder pro Sekunde⁶⁷ mit entsprechender Dauer erzeugt. Um Unterschiede auszuschließen, die aufgrund des Neu-Renderings entstehen könnten, wird das Originalvideo⁶⁸ ebenfalls erneut gerendert. Hier wird aber der Umrechnungsfaktor ‘1’ genommen, da wir dieses Video nicht ändern wollen. Die Auflösung im Vergleich zum Original wurde in allen neuen Videos nicht geändert. Um nun zu überprüfen, ob trotz des gleichen Videos, nur mit anderer Bildfrequenz, ein Unterschied in den Zeiten entsteht, wird das *ONNX* Netz zur Detektion verwendet. Es wird aber erwartet, dass die Zeiten sich gleichen, da jedes mal die gleiche Anzahl an Bildern verarbeitet werden muss. Die Anwendung wird vor dem ersten Laden des Videos neu gestartet. Das Starten der Anwendung erfolgt immer über den *Mu Editor*. Alle darauf folgenden Versuche mit dem gleichen Video werden ohne Neustart der Anwendung durchgeführt. In zehn Versuchen ergeben sich die in Tabelle 6.1 zu sehenden Zeiten⁶⁹.

Während der Analyse war die *CPU* immer fast komplett ausgelastet. Die *GPU* wurde nicht genutzt. Die *RAM*-Nutzung hat sich mit jeder Ausführung erhöht und auch der ‘Leerlauf’ der Anwendung hatte nach einer Ausführung eine höhere *RAM*-Nutzung als zuvor. Nach zehn Versuchen wurden hier aber noch unter 800 MB *RAM* während der Analyse verwendet.

Tabelle 6.1: Überprüfung des Einflusses der Bildrate

Versuch	Original (gerendert)	15 FPS	30 FPS	60 FPS
1	20,363790s	20,417074s	20,391375s	20,456005s
2	20,097423s	20,159681s	20,356238s	20,274630s
3	20,152045s	20,301532s	20,244416s	20,299293s
4	20,035980s	20,190585s	20,300770s	20,182451s
5	20,055424s	20,136865s	20,259396s	20,335312s
6	20,062323s	20,134515s	20,308602s	20,320889s
7	20,079172s	20,157559s	20,268122s	20,233600s
8	20,174469s	20,169565s	20,247477s	20,299185s
9	20,343245s	20,309957s	20,319228s	20,337153s
10	20,252515s	20,121260s	20,247946s	20,340947s
Durchschnittszeit	20,1616386s	20,2098593s	20,294357s	20,3079465s
Standardabweichung	0,120287s	0,098581s	0,050399s	0,072509s

Abbildung 6.39 zeigt die Durchschnittszeiten der verschiedenen Videos wie in Tabelle 6.1 ermittelt. Wie sich in Tabelle 6.1 bzw. Abbildung 6.39 erkennen lässt, hat die Frequenz der Bildrate wie erwartet keinen Einfluss auf die Laufzeit der Analyse. Daher kann in Folge mit und nur mit dem ungerenderten Original fortgeführt werden.

Der nächste Test bezieht sich auf die Leistung von verschiedenen neuronalen Netzen. Die Ursprungsnetze sind jeweils als *PyTorch*-Datei vorgegeben, eines für die Detektion und eines für die Segmentierung. Zusätzlich liegen diese beiden neuronalen Netze kon-

⁶⁷Entspricht einem Umrechnungsfaktor von 0,563

⁶⁸26.662 FPS

⁶⁹in Sekunden

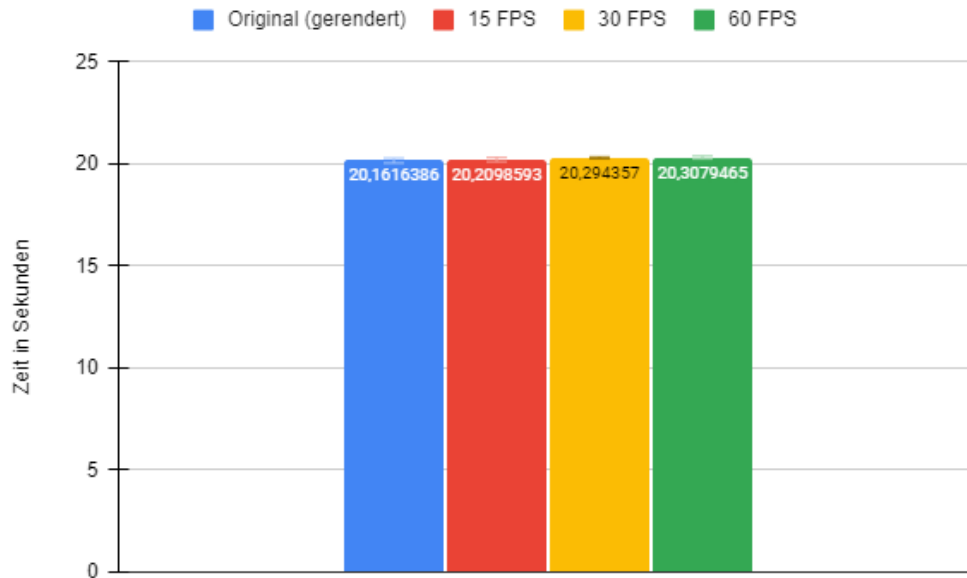


Abbildung 6.39: Graphische Darstellung der Durchschnittszeiten von Tabelle 6.1 mit Standardabweichung

vertiert nach *ONNX* vor. In diesem Test werden die Zeiten der verschiedenen neuronalen Netze (Segmentierung und Detektion, jeweils im *PyTorch*- und *ONNX*-Format) genommen und nicht von verschiedenen Videos. Die Umgebung und der Versuchsaufbau sowie die Durchführung sind dabei die selben wie beim Versuch zuvor. Tabelle 6.2 beschreibt die Laufzeiten der Anwendung bei der Nutzung verschiedener neuronaler Netze.

Tabelle 6.2: Überprüfung des Einflusses des neuronalen Netzes

Versuch	<i>ONNX</i> Det.	<i>ONNX</i> Seg.	<i>PyTorch</i> Det.	<i>PyTorch</i> Seg.
1	20,510654s	43,295197s	19,768659s	23,639113s
2	20,373760s	42,991104s	19,835788s	23,475227s
3	20,153060s	44,370268s	19,641468s	23,439868s
4	20,272583s	44,492352s	20,543483s	23,474091s
5	20,236375s	43,861180s	19,628065s	23,352280s
6	20,161826s	44,588593s	20,190879s	23,363823s
7	20,210409s	44,363141s	19,697228s	23,493183s
8	20,240074s	43,039226s	19,646677s	23,425029s
9	20,357902s	44,764669s	19,924803s	23,425241s
10	20,199726s	44,266812s	20,011906s	23,480054s
Durchschnittszeit	20,2716369s	44,0032542s	19,8888956s	23,4567909s
Standardabweichung	0,111694s	0,664212s	0,294021s	0,080002s

Abbildung 6.40 zeigt die Durchschnittszeiten der verschiedenen neuronalen Netze wie in Tabelle 6.2 ermittelt. In Tabelle 6.2 bzw. 6.40 lässt sich erkennen, dass das *ONNX*-Netz und das *PyTorch*-Netz zur Detektion ungefähr die gleiche Laufzeit haben und hier das Format einen kaum wahrnehmbaren Unterschied aufweist. Ebenso lässt sich erkennen, dass die neuronalen Netze zur Segmentierung länger brauchen als zur Detektion.

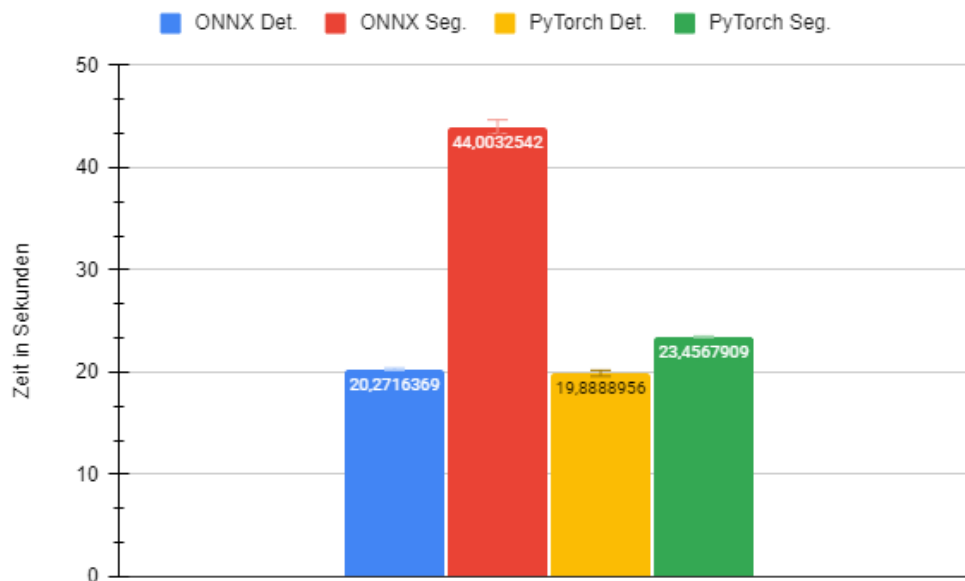


Abbildung 6.40: Graphische Darstellung der Durchschnittszeiten von Tabelle 6.2 mit Standardabweichung

Ein Grund dafür könnte sein, dass YOLO besser in der Detektion als in der Segmentierung ist [102]. Da ich die Netze allerdings nicht trainiert habe sondern nur zur Verfügung gestellt bekam, kann ich hier keine weiteren Mutmaßungen äußern. Deutlicher wird der Unterschied, wenn man sich das *ONNX*-Netz und das *PyTorch*-Netz zur Segmentierung vergleicht. Dabei hat das *ONNX*-Netz fast die doppelte Laufzeit wie sein *PyTorch* Pendant. Da ich bereits erwähnt hatte, dass in der Anwendung die Segmentierung mit *ONNX* zwar durchläuft, aber die Pupille nicht erkennt, könnte auch die deutlich abweichende Laufzeit durch ein Problem mit dem *ONNX*-Netz erklärbar sein.

Wie mir während der ersten Versuche aufgefallen ist, erhöht sich die *RAM*-Nutzung mit jeder Analyse. Deswegen werde ich nochmal zehn Analysen mit einem neuronalen Netz durchführen. Dafür wird das Originalvideo genommen und mit dem *ONNX*-Netz zur Detektion analysiert. Gemessen wird die *RAM*-Nutzung während und nach der Analyse. 'Nach der Analyse' beschreibt dabei den *idle* Zustand der Anwendung mit einem ausgewählten Video und einem ausgewählten neuronalen Netz. Direkt nach Start der Anwendung, also ohne geladenes Video und ohne geladenes neuronales Netz hat sie eine *RAM*-Nutzung von ca. 39,9 MB, mit geladenem Video und geladenem neuronalen Netz von ungefähr 47,0 MB. Da die Werte nicht konstant sind, wird versucht möglichst das Maximum aus dem *Task Manager* herauszulesen. Die Ergebnisse werden in Tabelle 6.3 festgehalten.

Abbildung 6.41 stellt die *RAM*-Nutzung aus Tabelle 6.3 graphisch dar mit Trendlinie. Bereits in Tabelle 6.3 fällt auf, dass die *RAM*-Nutzung während und nach der Analyse ansteigt. Außerdem fällt auf, dass Analyse 6 nicht in die Reihe passt. Dies kann aber nur Zufall sein. Dank der Trendlinien in Abbildung 6.41 erkennt man außerdem, dass die

Tabelle 6.3: Überprüfung der *RAM*-Nutzung der *PyQt6* Anwendung während und nach der Analyse

Analyse	Während der Analyse	Nach der Analyse
1	381,8 MB	351,9 MB
2	447,1 MB	416,7 MB
3	451,4 MB	419,4 MB
4	516,3 MB	485,0 MB
5	579,9 MB	548,3 MB
6	576,8 MB	545,2 MB
7	645,0 MB	614,7 MB
8	708,5 MB	671,6 MB
9	768,0 MB	735,3 MB
10	830,7 MB	800,7 MB

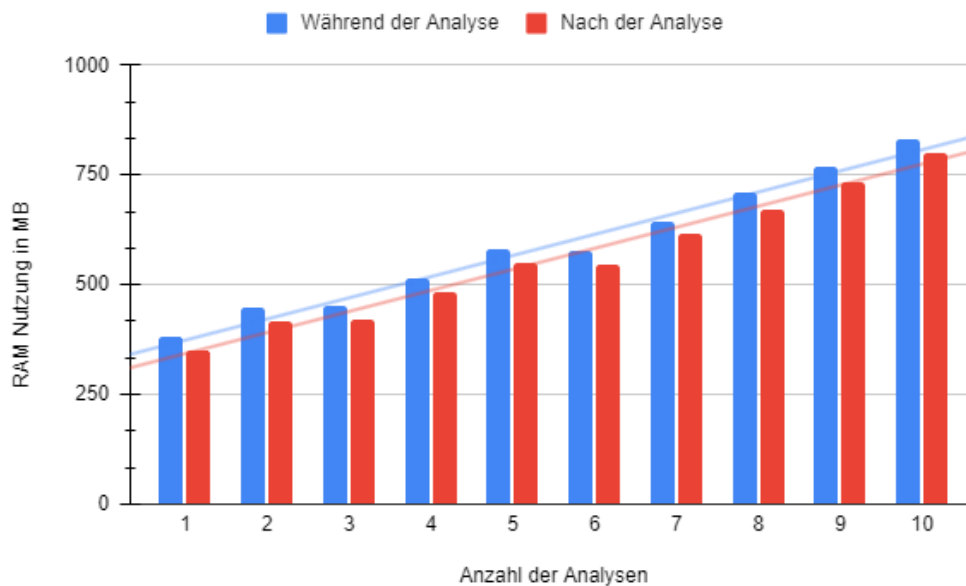


Abbildung 6.41: Graphische Darstellung der *RAM*-Nutzung von Tabelle 6.3 mit Trendlinie

RAM-Nutzung während und nach der Analyse ungefähr gleichstark ansteigt und bis auf Analyse 6 auch alle Ergebnisse zur jeweiligen Trendlinie passen. Ebenso stellt Analyse 10 einen Widerspruch zur vorherigen Aussage dar, dass während der zehnten Analyse noch unter 800 MB *RAM* genutzt wurden. Da ich jedoch nicht genügend Kenntnisse in der Speicherverwaltung von *Python* habe, kann ich hier keine Aussagen darüber treffen, warum die *RAM*-Nutzung mit jeder Analyse ansteigt und die Versuchsreihe eine andere *RAM*-Nutzung geliefert hat als die ursprüngliche Beobachtung.

Referenzsystem

Auf dem Referenzsystem werden die gleichen Tests mit der gleichen Durchführung durchgeführt wie auf dem Entwicklungssystem. Da bereits gezeigt wurde, dass die Anzahl an Bildern pro Sekunde eines Videos keinen Einfluss auf die Verarbeitungsdauer hat und zudem identisch damit ist, wenn man das Originalvideo verwendet, wird dieser Test nicht

erneut durchgeführt.

Damit wird hier als erster Test die Zeiten der verschiedenen neuronalen Netze miteinander verglichen. Gleichzeitig wird die *RAM*-Nutzung während des Tests mit dem *ONNX*-Netz zur Detektion festgehalten, da beide Tests den gleichen Ablauf haben. Die Ergebnisse der Zeiten werden in Tabelle 6.4 und Abbildung 6.42 festgehalten, die Ergebnisse der *RAM*-Nutzung in Tabelle 6.5 und Abbildung 6.43.

Vorweg sei gesagt, dass die *RAM*-Nutzung auf dem Referenzsystem nach Start bei 47,0 MB lag und nach Laden eines neuronalen Netzes bei 57,2 MB und damit ca. 10 MB höher als auf dem Entwicklungssystem. Auch wenn dies prozentual einen großen Unterschied darstellt, zumindest wenn man in MB rechnet, ist in einem realen Umfeld der Unterschied nicht spürbar und somit vernachlässigbar.

Da ich bei der dritten Versuchsreihe festgestellt habe, dass die Anwendung auf dem Referenzsystem nur noch 50-66% der *CPU* nutzte und nicht mehr 90-100%, habe ich mich dazu entschieden hier zusätzlich zwischen jeder Versuchsreihe das Referenzsystem für ein paar Minuten herunterzufahren und abkühlen zu lassen.

Tabelle 6.4: Überprüfung des Einflusses des neuronalen Netzes auf dem Referenzsystem

Versuch	<i>ONNX</i> Det.	<i>ONNX</i> Seg.	<i>PyTorch</i> Det.	<i>PyTorch</i> Seg.
1	39,036953s	68,176868s	38,608333s	43,221634s
2	33,738517s	66,900585s	33,638124s	40,705562s
3	34,056469s	78,908915s	33,991009s	42,190650s
4	34,023082s	80,305914s	34,387230s	40,896865s
5	33,928684s	79,925705s	34,165910s	45,032872s
6	33,999341s	73,969538s	33,482710s	45,772451s
7	35,116674s	68,043697s	34,460550s	46,929895s
8	36,123860s	76,648049s	34,304654s	52,773398s
9	38,456427s	85,947691s	35,490906s	51,361930s
10	46,839062s	84,521936s	37,109345s	48,982016s
Durchschnittszeit	36,5319069s	76,3348898s	34,9638771s	45,7867273s
Standardabweichung	4,106926s	6,869567s	1,657240s	4,232443s

Wie zu erwarten braucht die Analyse auf dem Referenzsystem grundsätzlich länger als auf dem Entwicklungssystem. Zu erwarten ist dieses Ergebnis daher, dass dem Entwicklungssystem deutlich mehr Leistung zur Verfügung steht. Ebenfalls brauchen die Analysen mit den neuronalen Netzen zur Detektion ungefähr gleich lange, Segmentierung dauert entscheidend länger.

In Tabelle 6.5 fällt auf, dass nach der fünften Analyse ein Einbruch der *RAM*-Nutzung auftritt. Ein ähnliches Phänomen gab es bereits auf dem Entwicklungssystem, nur blieb da die *RAM*-Nutzung zwischen zwei Versuchen konstant. Auch wenn die Trendlinie in Abbildung 6.43 aufgrund dieses Einbruchs nicht so sinnvoll wirkt wie in Abbildung 6.41, lässt sich dennoch beobachten, dass die *RAM*-Nutzung während und nach der Analyse

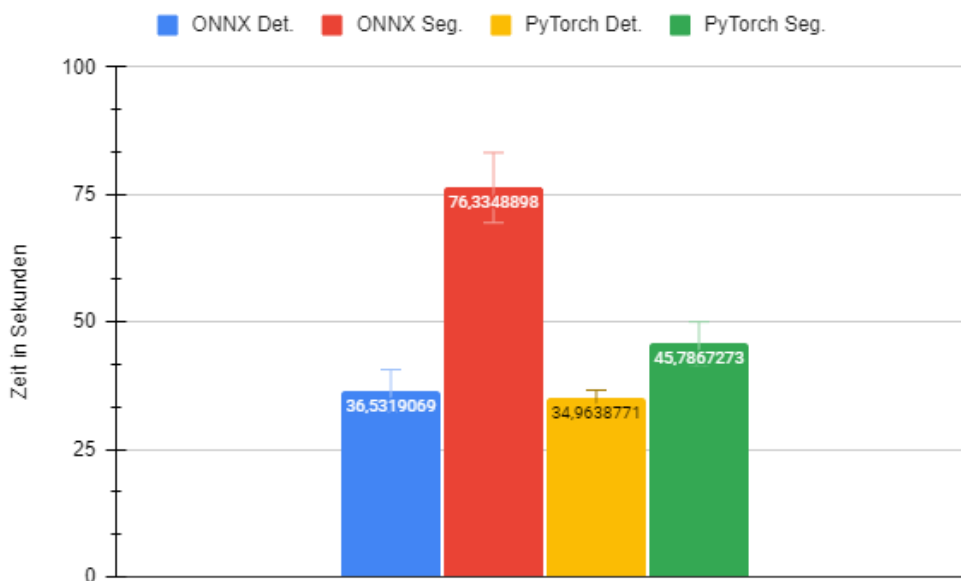


Abbildung 6.42: Graphische Darstellung der Durchschnittszeiten von Tabelle 6.4 mit Standardabweichung

Tabelle 6.5: Überprüfung der *RAM*-Nutzung der *PyQt6* Anwendung während und nach der Analyse auf dem Referenzsystem

Analyse	Während der Analyse	Nach der Analyse
1	380,9 MB	359,2 MB
2	445,5 MB	428,2 MB
3	510,4 MB	493,1 MB
4	573,9 MB	556,0 MB
5	637,5 MB	619,7 MB
6	447,8 MB	430,0 MB
7	516,6 MB	497,5 MB
8	576,8 MB	558,8 MB
9	638,0 MB	620,4 MB
10	698,4 MB	675,9 MB

parallel zueinander ansteigt.

Eine wirklich neue Erkenntnis ist hier aber die Kühlung des Systems. Da die Analyse zwischen 90-100% der *CPU* auslastet, kann es sein, dass im Dauerbetrieb die *CPU* von der Leistung her gedrosselt wird, damit das System nicht überhitzt. Eine andere Möglichkeit wäre, dass sich das System einfach abschaltet. In der Regel sollte ein Desktop-PC aber eine ausreichende Kühlung verbaut haben, weshalb es da keine Probleme geben sollte. Benutzt man hingegen ein Tablet wie in diesem Fall ein Microsoft Surface, kann die Kühlung nicht unbedingt dauerhaft ausreichen. Dieser Fall sollte aber nur eintreten, wenn man entweder eine Langzeitanalyse macht (z.B. aufgrund der Aufnahme mit einer Kamera) oder viele Analysen direkt aneinander reiht.

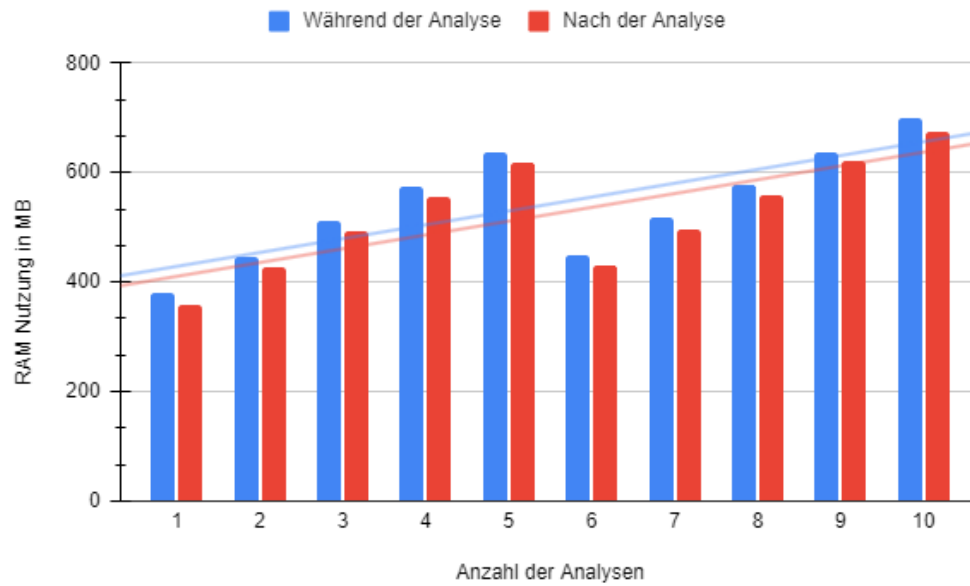
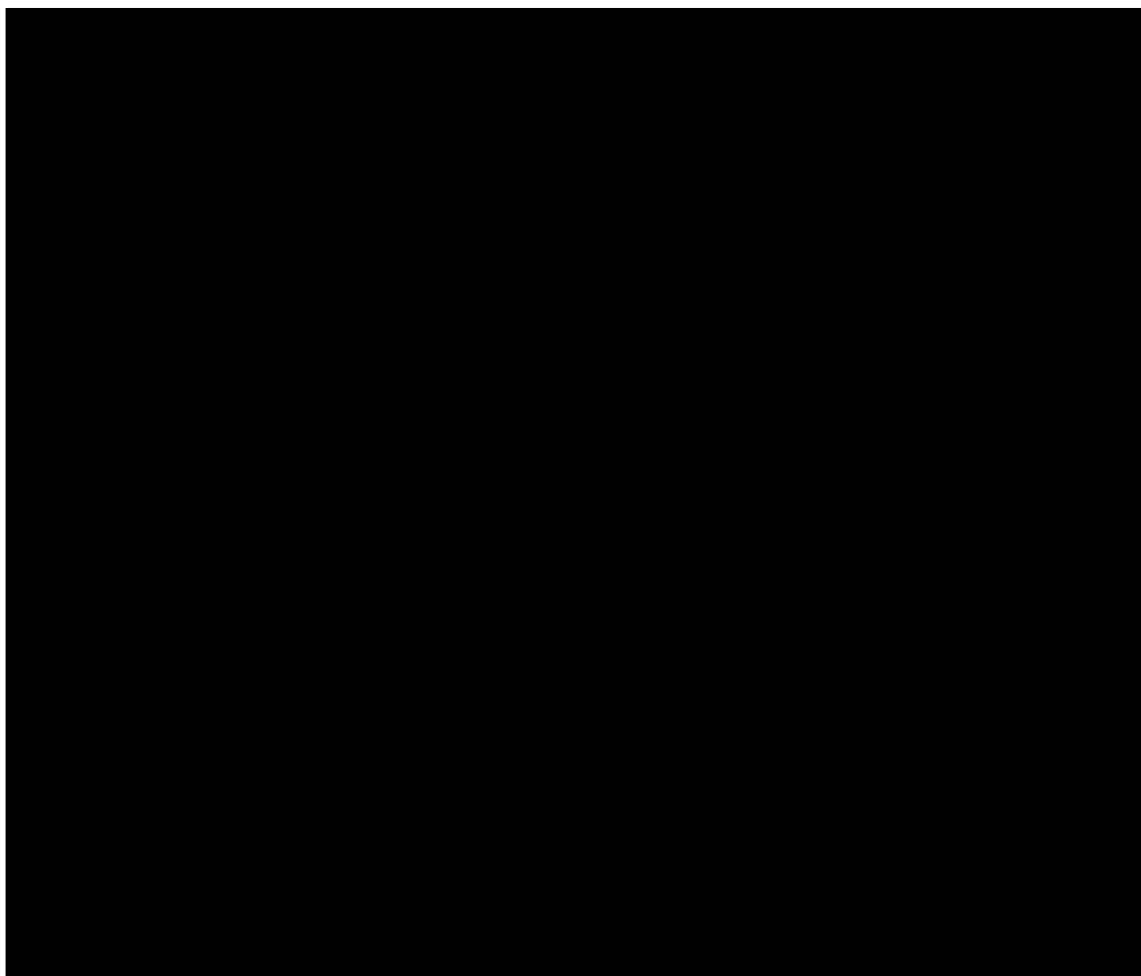


Abbildung 6.43: Graphische Darstellung der *RAM*-Nutzung von Tabelle 6.5 mit Trendlinie

6.4 JavaFX



Ergebnisse

7.1	Handhabung der Werkzeuge	121
7.1.1	Einrichten und Handhaben der IDEs	121
7.1.2	Dokumentation und Tutorials	124
7.1.3	Entwicklung der Anwendung	125
7.2	Performanz der Anwendungen	130

Dieses Kapitel fasst die Ergebnisse und Erfahrungen aus Kapitel 6 zusammen und vergleicht diese miteinander. Wichtig dabei ist, dass gerade nachfolgendes Kapitel zur Handhabung hauptsächlich auf persönlichen Empfinden basiert.

7.1 Handhabung der Werkzeuge

Meine Eindrücke habe ich versucht in Tabelle 7.1 zusammenzufassen und gegenüber zu stellen. Daher steht diese Tabelle auch in Bezug zu allen Aussagen, die ich hier auch in reiner Textform treffen werde, ohne speziell nochmal auf die Tabelle einzugehen.

7.1.1 Einrichten und Handhaben der IDEs

Beginnen möchte ich mit dem Einrichten der jeweiligen IDEs.

Das Einrichten der IDE für *UWP* habe ich als recht einfach empfunden, da dies auch gut beschrieben war. Das einzige Problem war das Schließen und neu Aufrufen des *Installer* über den Konsolenbefehl, aber dies konnte ich umgehen. Andernfalls sollte man darauf aufpassen. Ich denke jedoch, dass man im Normalfall den *Installer* nicht schließt. Etwas schwieriger kann die Arbeit sein, wenn es darum geht auch die richtigen *Packages* zu installieren. Wenn man da keine Anleitung hat und nicht genau weiß, wonach man sucht, kann das mitunter ein etwas schwierigeres Unterfangen werden.

Tabelle 7.1: Überblick über meine persönlichen Eindrücke und Erfahrungen der drei ausgewählten Werkzeuge

Positiv	Neutral	Negativ
UWP		
Ausführliche Dokumentation	<i>Hello World</i> Tutorial lösbar, ohne selbst zu programmieren	Dokumentation auf <i>C#</i> ausgelegt
<i>Frontend</i> und <i>Backend</i> in einer IDE anpassbar	Einrichten der IDE kann umständlich sein	Einbinden externer Bibliotheken war nicht möglich
Oberflächenelemente via Menü editierbar	Anzahl an Tutorials eher gering	
	Vorlagencodes ausführlich aber teils komplizierter als nötig	
	Es wird Wissen um <i>C++</i> vorausgesetzt (Bsp. Asynchrone Prozesse)	
Qt		
Einrichten einer Entwicklungsumgebung sehr einfach	In den <i>Communitytutorials</i> muss man nicht programmieren	
<i>Mu Editor</i> war gute Unterstützung		
<i>Qt Designer</i> leicht zu bedienen		
Sehr umfangreiches <i>Communitytutorial</i>		
Innhalb kurzer Zeit lässt sich eine gute Anwendung schreiben		
JavaFX		
Einrichten aller Entwicklungsumgebungen aufgrund Vorkenntnisse sehr einfach	<i>JFX Scene Builder</i> Tutorials sind stark veraltet	Offizielle Tutorials beschränken sich auf <i>Getting Started</i>
<i>JFX Scene Builder</i> sehr umfangreich	<i>JFX Scene Builder</i> ist mehr <i>learning-by-doing</i>	
<i>Communitytutorials</i> geben guten Überblick	<i>Multithreading</i> kann zu Problemen führen	

Was an *Visual Studio 2022 Community Edition* während der Entwicklung meines Erachtens nach positiv hervorzuheben war, ist, dass die Oberfläche, sofern man eine gültige *.xaml* Datei hat, direkt in der IDE angezeigt wurde. Dadurch konnte man immer sehen, welche Änderung genau was getan hat, ohne dafür extra die Anwendung kompilieren und starten zu müssen. Dadurch fiel natürlich auch die Notwendigkeit des Nutzens eines

externen Werkzeugs, um das Bauen der Oberfläche zu vereinfachen, weg. Die meisten Einstellungen, wenn nicht gar alle, für ein Oberflächenelement konnte man ebenfalls per extra Menü auf Knopfdruck hinzufügen, entfernen oder ändern. Das erspart sonst notwendige Codier- und Recherchearbeit.

Ebenfalls vorteilhaft ist, dass der *Projektmappen-Explorer* alle notwendigen Dateien¹ für die Oberfläche unter der jeweiligen *.xaml* zusammenfasst. Dadurch gewinnt das Projekt deutlich an Übersicht.

Was sich hingegen deutlich schwieriger gestaltete, war das Einbinden von externen Bibliotheken. Dies ist mir trotz einer Recherche, die sich gefühlt irgendwann im Kreis gedreht hat, und mehrtägigen rumprobieren nicht gelungen. Ich vermute, dass es sich hierbei um ein IDE Problem handelt, ich möchte es aber auch nicht ausschließen, dass ich irgendwas falsch gemacht habe.

Deutlich einfacher war das Einrichten einer IDE für *PyQt6*, da man dies theoretisch auch über den *Texteditor* hätte machen können. Alles worauf man achten musste, war, die richtigen Bibliotheken installiert zu haben. Das Nutzen des *Mu Editors* hat die Entwicklung auf jeden Fall nochmal vereinfacht. Diese IDE hat mir nicht nur das Ausführen über eine Konsole abgenommen, da man in der IDE einfach auf den Startbutton klicken konnte, sondern auch das Installieren der Bibliotheken. Dafür musste ich nur die gewünschte Bibliothek als externe Abhängigkeit eintragen, und die IDE hat mir automatisch diese Bibliothek sowie alle dafür notwendigen Bibliotheken heruntergeladen. Neben der Syntaxhervorhebung konnte der *Mu Editor* auch den Code automatisch einrücken sowie ihn auf Inkonsistenzen prüfen. Dazu gehört als einfaches Beispiel, dass Variablen markiert werden, welche zwar an einer Stelle benutzt, aber vorher nie deklariert wurden.

Aber auch das Einrichten des *Qt Designers* war sehr einfach, da dies über einen normalen *Installer* geschah.

Sowohl das Nutzen des *Mu Editors* und des *Qt Designers* war sehr intuitiv. Beide Programme sind sehr übersichtlich gehalten. Gerade beim *Mu Editor* merkt man, dass dieser sich nur auf die Kernelemente einer IDE fokussiert und alles mitbringt, was man wirklich braucht, und nicht, was man brauchen könnte. Der *Qt Designer* ist ebenfalls sehr übersichtlich in einfach in der Handhabung. Innerhalb kürzester Zeit war ich in der Lage die entsprechenden am Ende zu sehenden Oberflächen zusammenzubauen.

Einen Vorsprung in die Richtung hatte die Umgebung für *JavaFX*. *Eclipse* als IDE war mir bereits bekannt, da ich schon damit gearbeitet habe. Ebenso das Installieren der entsprechenden *Java* Version. Zusätzlich kam noch der *JavaFX Scene Builder* hinzu, der ebenfalls wie der *Qt Designer* mithilfe eines *Installers* installiert wurde.

In der Handhabung ähnelt *Eclipse Visual Studio 2022 Community Edition*. Es ist ebenfalls eine sehr umfangreiche IDE, welche sehr vieles mitbringt. Gerade das Erstellen des Vorlageprojekts für die *JavaFX* Anwendung war eine große Hilfe, um die Anwendung sauber aufbauen zu können.

¹.*cpp*, *.h* und *.idl*

Die Vorschau und der Zusammenbau der Oberfläche übernahm der *JavaFX Scene Builder*. Dieser ist im Prinzip aufgebaut wie der *Qt Designer* bietet aber eine größere Auswahl an vordefinierten Objekten. Daher konnte ich an dieser Stelle aufgrund der vorher gesammelten Erfahrungen besser arbeiten.

Zusammenfassend ist in dem Bereich des Einrichtens der jeweiligen IDE meines Empfindens nach keiner besser als der andere. In der Handhabung hingegen fand ich persönlich die Mischung aus dem *Mu Editor* und dem *Qt Designer* am angenehmsten und potentiell am einsteigerfreundlichsten.

7.1.2 Dokumentation und Tutorials

Als nächstes widme ich mich der vorhandenen Dokumentation und den angebotenen Tutorials.

UWP kommt selbst tatsächlich mit eher wenig Tutorials, zumindest wenn es um den Bereich *Erste Schritte mit UWP* geht. Gerade dass diese Tutorials in *C#* gehalten sind, macht den Einstieg in die *C++* Variante nicht einfacher. Dadurch war es mir mit meinen begrenzten Vorkenntnissen in *C++* nicht möglich, alle diese Tutorials zu bearbeiten. Die *Hello World* Anwendung ist zwar nett, um überhaupt ein funktionierendes Gerüst zu haben, jedoch muss man hier nichts programmieren. Die anderen Tutorials hingegen erfordern Programmierkenntnisse, gerade wenn man den vorgegebenen Code in *C++* übersetzen möchte. Aber wenn etwas nicht funktioniert, ist es schwierig zu verstehen, warum. Allerdings gibt es auch eine umfassende Einführung in *UWP/C++*. Dort werden allerdings eher Konzepte erklärt als wirklich Tutorials gegeben. Allgemein scheint sich *UWP* eher auf erfahrenere Entwickler zu konzentrieren, wobei es mit *C#* potentiell einfacher ist als mit *C++*, da die Dokumentation darauf ausgelegt ist.

Zu *PyQt6* gibt es ein sehr umfangreiches *Communitytutorial*, dessen Abarbeitung auch viel Zeit in Anspruch genommen hat. Ich kann aber sagen, dass sich dieses für mich definitiv gelohnt haben und Entwickler unabhängig ihrer Vorkenntnisse abholt. Es werden sowohl grundlegende Prinzipien erklärt, als auch ein Lösungsweg für gängige Aufgabenstellungen gegeben. Dass in jedem Tutorial der gesamte Lösungscode gegeben und erklärt wird, und in der Regel auch funktioniert, hilft zwar dabei zu sehen und zu verstehen, wie man ein Problem löst, aber nicht unbedingt dabei das Programmieren zu erlernen.

Für *JavaFX* ist es unerlässlich Kenntnisse in *Java* zu haben. Die *Getting Started with JavaFX* Dokumentation beschreibt nur, wie man in einer gewählten Umgebung ein neues *JavaFX* Projekt aufsetzen kann. Definitiv hilfreich dabei ist, dass dieses Projekt kompilierbar und ausführbar ist. Denn wenn die Anwendung plötzlich nicht mehr funktioniert, dann weiß man, dass man etwas falsch gemacht hat. Um aber wirklich die Prinzipien von *JavaFX* zu erlernen, benötigt man die dort verlinkten *Communitytutorials*. Mithilfe dieser *Communitytutorials* versteht man zwar die Prinzipien von *JavaFX*, sie sind aber weniger

ausführlich wie das *Communitytutorial* von *PyQt6* und bedürfen teilweise auch nur Codeabschnitte, ähnlich wie bei *UWP*.

Tutorials zum *JavaFX Scene Builder* sind dazu sogar schon teilweise stark veraltet, sodass diese nur hilfreich sind, um die Oberfläche zu verstehen. Den *JavaFX Scene Builder* erlernt man also eher durch *learning-by-doing*, anstatt an kleineren Projekten.

Zusammenfassend sind für mich die Dokumentation und die Tutorials für *PyQt6* am hilfreichsten gewesen. Mit dieser Aussage möchte ich aber die anderen Dokumentationen und Tutorials nicht schlecht reden, sondern lediglich *PyQt6* positiv hervorheben. Klassen- und Methodendokumentation sind im Übrigen für alle drei Werkzeuge vollständig vorhanden gewesen, weswegen ich den Vergleich dieser nicht mit aufgenommen habe.

7.1.3 Entwicklung der Anwendung

Zuletzt möchte ich noch auf die Entwicklung der eigentlichen Anwendung eingehen.

Die Entwicklung der Anwendung mit *UWP* empfand ich am schwierigsten von den drei gewählten Werkzeugen. Das lag zum einen daran, dass sich meine *C++*-Erfahrung auf ein Lehrfach während meines Bachelorstudium reduziert und auch dieses schon ein paar Jahre her ist. Zum anderen lag es daran, dass die Dokumentation auf *C#* ausgelegt ist und ich vieles mühsam zusammensuchen musste, wovon auch nicht alles funktionierte. Außerdem sind die in der Dokumentation gegebenen Codebeispiele zwar ausführlich, aber teilweise lassen sich Sachen deutlich einfacher realisieren. Ein Beispiel davon ist die Navigation zwischen den Seiten. Auch das notwendige asynchrone Warten für den *FilePicker* war durchaus eine Herausforderung zu verstehen. Dennoch hat gerade die Entwicklung der Oberfläche innerhalb der IDE mir viel Freude bereitet. Auch optisch finde ich die mit *UWP* entwickelte Anwendung durchaus ansprechend, auch wenn ich es nicht geschafft habe, die Anwendung zu Ende zu entwickeln. Das lag aber hauptsächlich an dem in Kapitel 6.2.2 beschriebenen Problem.

Eine sehr angenehme Entwicklungserfahrung hatte ich mit *PyQt6*, auch wenn ich vorher nur ganz grundlegende *Python*-Kenntnisse hatte. Das lag hauptsächlich an dem sehr ausführlichen *Communitytutorial*, wodurch ich letzten Endes mehr gelernt hatte, als ich wirklich brauchte. Die Oberflächen waren mit dem *Qt Designer* schnell geschrieben. Zwar musste ich lernen, dass es für die Navigation ausreicht nur ein Fenster zu haben, und die verschiedenen Seiten als eigenes *Widget* zu definieren, aber glücklicherweise konnte man im *Qt Designer* den Seitenaufbau einfach in ein *Widget* kopieren, was mir einige Minuten ersparen konnte.

Natürlich war ebenso hilfreich, dass ich die Skripte für Detektion und Segmentierung als Vorlage erhalten hatte und mich an diesen orientieren konnte. Tatsächlich funktionierte hier alles überraschend schnell, sodass die Anwendung sogar innerhalb weniger Tage testbereit war.

Das Entwickeln der *JavaFX*-Anwendung war zumindest vom Prinzip her durchaus simpel. Dass das Vorlagenprojekte mit zwei Seiten kam, zwischen denen man hin und her navigieren konnte, übernahm direkt das Implementieren der Navigation, wodurch ich nur die richtigen Seiten einbinden musste. Die größte Herausforderung war aber tatsächlich das notwendige *Multithreading*. Das liegt daran, dass die Analyse des neuronalen Netzes nicht auf dem *JavaFX Thread* laufen darf, da sonst die Oberfläche nicht aktualisiert werden kann. Die Aktualisierung der Oberfläche funktioniert nur zuverlässig, wenn diese über den *JavaFX Thread* läuft. Durch das *Multithreading* kam aber die Bilderdarstellung nicht mehr hinterher. Ein Problem, was ich bis zum Ende nicht lösen konnte.

Da ich im gesamten Kapitel 6 nicht wirklich auf Quellcode eingegangen bin, möchte ich an dieser Stelle nochmal kurz zwei Aspekte beleuchten. Zum einen die Navigation zwischen den Seiten, zum anderen die Dateiauswahl. Die Analyse mit dem neuronalen Netz möchte ich an dieser Stelle außen vor lassen, da in allen drei Anwendungen *OpenCV* verwendet wird, bzw. verwendet werden sollte, und somit die Unterschiede marginal sind. Den ungefähren Aufbau der Skripte kann man als Pseudocode der *PyQt6* Anwendung in Listing 6.6 nachlesen.

Da ich es eben zuerst genannt habe, werde ich mit der Navigation auch direkt beginnen. Die Navigation in *C++* ist über ein Navigationsmenü implementiert. Im Vergleich dazu stehen die Navigationen aus *PyQt6* und *JavaFX*, wo ich es simplifiziert mit Buttons gelöst habe. Natürlich kann man die Navigation in *C++* auch über Buttons lösen. Genauso kann man die Navigation in *PyQt6* und *JavaFX* über Listen oder ähnliches implementieren.

Die Methode für die Navigation in *UWP*, welche in jeder Klasse implementiert werden muss, funktioniert wie in Listing 7.1 dargestellt. Zuerst wird von dem angeklickten Feld der in der *.xaml* definierte *Tag* abgelegt, da in Folge mehrfach auf diesen zugegriffen werden muss. Dann wird in einer Verkettung von *if*-Bedingungen geprüft, welcher *Tag* angeklickt wurde und zur entsprechenden Seite navigiert. Die Prüfung auf den *Tag* der aktuellen Seite wurde nicht definiert. Stattdessen wird der *Tag* in einem finalen *else* abgefangen, wo nichts getan wird. Damit ist gleichzeitig abgefangen, falls ein unbekannter *Tag* angeklickt wurde. Falls man hier die Navigation über Buttons lösen wollen würde, würde der Codeabschnitt innerhalb der *if*-Bedingung ausreichen. Wichtig dabei ist, dass sich alle Seiten über die jeweilige *Header-Datei*² kennen.

```
// Seiten-Klasse
void PupilAnalysisPage::navView_SelectionChanged( NavigationView
    const& sender, NavigationViewSelectionChangedEventArgs const&
    args )
{
    auto navItemTag = unbox_value<hstring>(args.
        SelectedItemContainer().Tag());
```

².h

```

    if (navItemTag == unbox_value<hstring>(
        navigationExperimentalPage().Tag())) {
        Frame().Navigate(TypeName(xaml_typename<ChiMockUwp::
            ExperimentalPage>())));
    }
    ...
}

```

Listing 7.1: Die Implementierung der Navigation in *UWP*

Verglichen damit läuft die Navigation in *PyQt6* etwas anders ab, wie in Listing 7.2 dargestellt. Durch den Luxus, dass man hier nur ein Fenster definiert hat, und die verschiedenen Seiten über ein *Widget* einbindet, muss die Navigation nur einmal in der *Main*-Klasse definiert werden. Die *Main*-Klasse muss natürlich die anderen Klassen, welche die *Widgets* und deren Logik implementieren, kennen. Die ersten zwei Codezeilen definieren dabei das Erstellen einer Seite in der *Main*-Klasse und wie man sie dem *StackWidget* hinzufügt, welche alle Seiten kennt. Anschließend wird der entsprechende Button mit der Methode verbunden, über welche die Navigation stattfinden soll. In dieser Methode wird der aktuelle Index des *StackWidgets* auf die Seite gestellt, zur welcher navigiert werden soll. Anstatt eine Referenz auf die Seite mitzugeben, kann man auch hart eine Zahl einbinden. Wenn man dann aber die Implementierungsreihenfolge der Seiten auf den *Stack* ändert, navigiert man zu den falschen Seiten.

```

// Main-Klasse
self.startPage = StartPage()
self.stackedWidget.addWidget(self.startPage)

self.navToDefault_button.clicked.connect(self.navigateToDefault)

def navigateToDefault(self):
    self.stackedWidget.setCurrentIndex(self.stackedWidget.
        indexOf(self.startPage))

```

Listing 7.2: Die Implementierung der Navigation in *PyQt6*

Die Navigation mit *JavaFX*, welche in 7.3 dargestellt ist, muss ähnlich wie bei *UWP* in jeder Klasse implementiert werden. Die *Controller*-Klassen implementieren nur eine Methode um zur entsprechenden Seite zu navigieren. Je nach Anzahl der Seiten muss diese Methode entsprechend oft implementiert werden. Die Methode ruft dabei eine Methode aus der *Main*-Klasse auf und erhält als Parameter den Pfad zu der Seite. Ich habe mich dabei dazu entschieden, die Pfade in einer eigenen *Enumeration*-Klasse abzulegen, da man so bei einer Veränderung der Dateistruktur die Navigation nur an einer Stelle warten muss.

Die von hier aufgerufene Methode aus der *Main*-Klasse setzt eine neue *Root* für die *Szene* und ladet dafür die übergebene *FXML*. Der Ladeprozess der *FXML* ist dabei in eine

andere Methode ausgelagert. Diese beiden waren auch bereits von dem Vorlagenprojekt implementiert, sodass ich an dieser Stelle nichts mehr tun musste.

```
// Controllerklasse
private void navigateToPupilAnalysis() throws IOException {
    App.setRoot(ENavigation.PUPILANALYSIS.getPath());
}

// Main-Klasse
public static void setRoot(String fxml) throws IOException {
    scene.setRoot(loadFXML(fxml));
}

private static Parent loadFXML(String fxml) throws IOException {
    FXMLLoader fxmlLoader = new FXMLLoader(App.class.getResource
        (fxml + ".fxml"));
    return fxmlLoader.load();
}
```

Listing 7.3: Die Implementierung der Navigation in *JavaFX*

Wie man erkennen kann, unterscheiden sich die Navigationsmethoden zwischen den verschiedenen Werkzeugen nicht großartig. Am Ende steht immer eine Codezeile, welche dafür sorgt, dass eine andere Seite aufgerufen wird. Das Gerüst darum unterscheidet sich jedoch von Werkzeug zu Werkzeug. Wobei bei einer Navigation rein über Buttons wohl *UWP* mit *C++* die einfachste Variante darstellt, da man hier wirklich nur eine einzige Codezeile für braucht, wenn man den Methodenrumpf außen vor lässt.

Eine andere Hürde pro Werkzeug war das Auswählen der Datei über den Dateibrowser des Betriebssystems zu implementieren. Listing 7.4 zeigt den Quellcode zur Dateiauswahl in *UWP*. In *UWP* und *C++* ist es wichtig, dass die entsprechende Methode einen Rückgabebetyp von *IAsyncOperation<StorageFile>* hat, da sonst die Methode nicht richtig funktioniert. Es ist aber nicht notwendig, dass die Methode auch etwas zurück gibt. In der ersten Zeile in der Methode wird der *FileOpenPicker*, wie die Klasse hier heißt, initialisiert. Als nächstes werden in der zweiten Zeile die entsprechenden Endungen hinzugefügt, welche der *FileOpenPicker* erkennen soll. Mit einem *** kann man auch alle Dateitypen erlauben. Danach wird in der dritten Zeile die Datei in einem asynchronen Prozess ausgewählt. Der Prozess der Dateiauswahl ist deshalb asynchron, weil hier nicht das Anwendungsfenster verwendet wird. Damit aber die Anwendung nicht weiterläuft, während der Dateibrowser geöffnet ist, ist das *co_await* notwendig. Durch dieses *Keyword* wird der Anwendung symbolisiert, dass er hier auf das Beenden eines anderen Prozesses warten soll.

```

IAsyncOperation<StorageFile> PupilAnalysisPage::
    actionPanelButtonSubmitLoadingNN_Click(IIInspectable const&
sender, RoutedEventArgs const& e)
{
    auto picker = FileOpenPicker();
    picker.FileTypeFilter().Append(L".onnx");
    const auto neuronal_network_file = co_await picker.
        PickSingleFileAsync();
    ...
}

```

Listing 7.4: Die Implementierung der Dateiauswahl in *UWP*

Im Vergleich dazu ist die Dateiauswahl in *PyQt6* deutlich einfacher gestaltet, wie 7.5 zeigt. Hier muss nur eine Methode definiert werden und die Dateiauswahl besteht im Prinzip aus einer Zeile, welche aus Gründen der Übersicht hier untereinander geschrieben ist. Alle Parameter, welche für den Dateibrowser notwendig sind, werden hier direkt beim Aufruf mitgegeben und nicht wie in *UWP* durch gesonderte Methoden modifiziert. Das erste Argument *self* symbolisiert den *Parent* des Dateibrowsers, also zu welchem Fenster oder *Widget* der Dateibrowser gehört. Dadurch kann die Anwendung im Hintergrund nicht weiter genutzt werden, bis der Dateibrowser wieder geschlossen wird. Als nächstes wird der Name des Fensters für den Dateibrowser mitgegeben. Hier soll der Dateibrowser als sein Name *Open File* anzeigen. Im Argument darauf wird der Startpfad des Dateibrowsers definiert, also welcher Ordner beim Öffnen des Dateibrowsers angezeigt werden soll. Ein leerer Parameter, wie es hier der Fall ist, symbolisiert den aktuellen Ordner der Anwendung als Startpfad. Das letzte Argument beschreibt die Dateitypen, welche ausgewählt werden können. Außerdem kann man den Dateitypen auch noch einen Anzeigenamen mitgeben, welcher bei der Auswahl mitgegeben werden kann. Auch hier können mit einem *** alle Dateitypen ausgewählt werden.

```

def load_nn(self):
    self.loadedNnFile = QFileDialog.getOpenFileName(
        self,
        "Open_File",
        "",
        "Neuronale_Netze_(*.onnx*.pt)",
    )

```

Listing 7.5: Die Implementierung der Dateiauswahl in *PyQt6*

Zuletzt möchte ich noch kurz die Dateiauswahl in der *JavaFX* Anwendung erklären, was in Listing 7.6 zu erkennen ist. Wichtig dabei ist, dass ich hier die Dateiauswahl in eine eigene Klasse ausgelagert habe und die Initialisierung der Klasse im Konstruktor vornehme. Verglichen kann das mit der ersten Zeile innerhalb der Methode der *UWP*-Anwendung. Die Methode hier muss aber im Vergleich zur *UWP*-Anwendung die Datei

auch zurückgeben. Als Parameter erhält die Methode ein Fenster. Dieses Fenster ist das *Parent*-Fenster der Anwendung. Im Normalfall ist das ein beliebiger Knoten der Anwendung, selbst ein Button kann im Endeffekt als Fenster übergeben werden. Wichtig ist dieses Fenster, damit die Anwendung einfriert, während der Dateibrowser geöffnet ist. Man kann als Parameter auch *null* oder ein neues Fenster übergeben, aber dann kann man die Anwendung im Hintergrund weiter benutzen, was zu Problemen führen kann. Die erste Zeile innerhalb der Methode definiert erstmal den Startpfad als *String*. Durch `.` wird signalisiert, dass der aktuelle Ordner der Anwendung als Startpfad genutzt werden soll. Darauf wird dem *FileChooser*, wie die Klasse in *JavaFX* heißt, der Startpfad übergeben. Anschließend werden die erlaubten Dateitypen, welche der *FileChooser* auswählen darf, modifiziert. Auch hier kann neben der Endung wieder ein Name mit übergeben werden, welcher stattdessen angezeigt wird. Zuletzt wird mithilfe von *showOpenDialog(window)* der Dateibrowser angezeigt.

```
public File selectFile(Window window) {
    String startDir = Paths.get(".").toAbsolutePath().normalize()
        .toString();
    fileChooser.setInitialDirectory(new File(startDir));
    fileChooser.getExtensionFilters().addAll(
        new ExtensionFilter("Neuronale_Netze", "*.onnx")),
    return fileChooser.showOpenDialog(window);
}
```

Listing 7.6: Die Implementierung der Dateiauswahl in *JavaFX*

Bei den drei Varianten erkennt man durchaus Ähnlichkeiten zwischen *UWP* und *JavaFX*, da in beiden Fällen jeder Parameter des entsprechenden Dateibrowsers einzeln modifiziert werden muss. *PyQt6* hingegen erlaubt es alle Parameter beim Öffnen des Dateibrowsers zu übergeben.

Abschließend möchte ich sagen, dass mir das Entwickeln der Anwendung mit allen drei Werkzeugen Spaß gemacht hat und es hier mehr auf persönliche Präferenzen und Vorkenntnisse ankommt, um zu entscheiden, was einem am besten liegt.

7.2 Performanz der Anwendungen

Nur weil einem eine Programmiersprache mehr liegt als die andere, nur weil man mit einem Werkzeug mehr Erfahrungen hat als mit anderen, heißt es noch lange nicht, dass basierend auf den zwei Aspekten die beste Wahl getroffen werden kann. Ein wichtiger Gesichtspunkt, den man auch betrachten muss, ist die Performanz der entwickelten Anwendungen. *UWP* kann ich an dieser Stelle nicht aufnehmen, da es Probleme beim Einbinden von *OpenCV* gab und so nie eine Performanz gemessen werden konnte. Daher konzentriert sich dieser Abschnitt auf den Vergleich zwischen *PyQt6* und *JavaFX*.

Da die Tests auf dem Referenzsystem die gleichen Erkenntnisse erbrachten wie die Tests

auf dem Entwicklungssystem, wird in Folge nur auf die Tests auf dem Entwicklungssystem Bezug genommen.

Das erste Experiment hat gezeigt, dass es nicht auf die Videolänge oder den Bildern pro Sekunde ankommt, sondern nur auf die Anzahl der Bilder, welches ein Video hat. Außerdem wurde im ersten Experiment ein neuronales Netz verwendet, welches auch im zweiten Experiment beim Vergleich zwischen den verschiedenen Netzen vor kam. Die Zeiten zwischen dem ersten und zweiten Experiment beim gleichen neuronalen Netz waren deckungsgleich, daher kann ich hier direkt mit dem zweiten Experiment beginnen. Abbildung 7.1 zeigt die Zeiten von der *PyQt6*-Anwendung und der *JavaFX*-Anwendung. Die Zeiten des neuronalen Netzes im *PyTorch*-Format lasse ich hier aus, da diese Messung nur in der *PyQt6*-Anwendung möglich war. Die Zeiten befanden sich aber auf einem sehr ähnlichen Niveau zu den Zeiten des *ONNX*-Netzes.

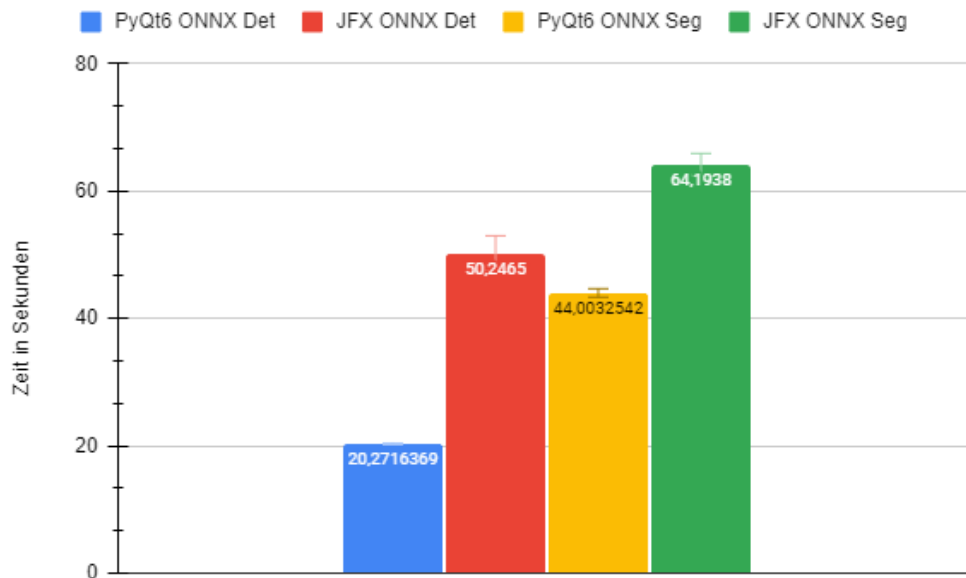


Abbildung 7.1: Vergleich der Durchschnittszeiten der verschiedenen neuronalen Netze bei verschiedenen Anwendungen

In Abbildung 7.1 kann man sehr gut erkennen, dass die *JavaFX*-Anwendung im Schnitt deutlich länger als die *PyQt6*-Anwendung braucht. In der Implementierung ist der einzige echte Unterschied, dass bei der *JavaFX*-Anwendung das neuronale Netz über eine *OpenCV*-Klasse geladen wurde, in *PyQt6* aber über *Ultralytics* und *YOLO*. Aber auch ohne das neuronale Netz zu implementieren, sondern nur das Video mithilfe von *OpenCV* einlesen zu lassen, dauert in *JavaFX* bereits länger. Ich kenne mich aber nicht gut genug sowohl in *Python* als auch in *Java* als auch in *OpenCV* aus, um Mutmaßungen darüber anzustellen, woher die Unterschiede kommen.

In puncto Geschwindigkeit hat die *PyQt6*-Anwendung einen Vorteil. Als nächstes steht aber noch der Vergleich der *RAM*-Nutzung an. Abbildung 7.2 stellt die beiden Anwen-

dungen in deren zehn Durchläufen gegenüber.

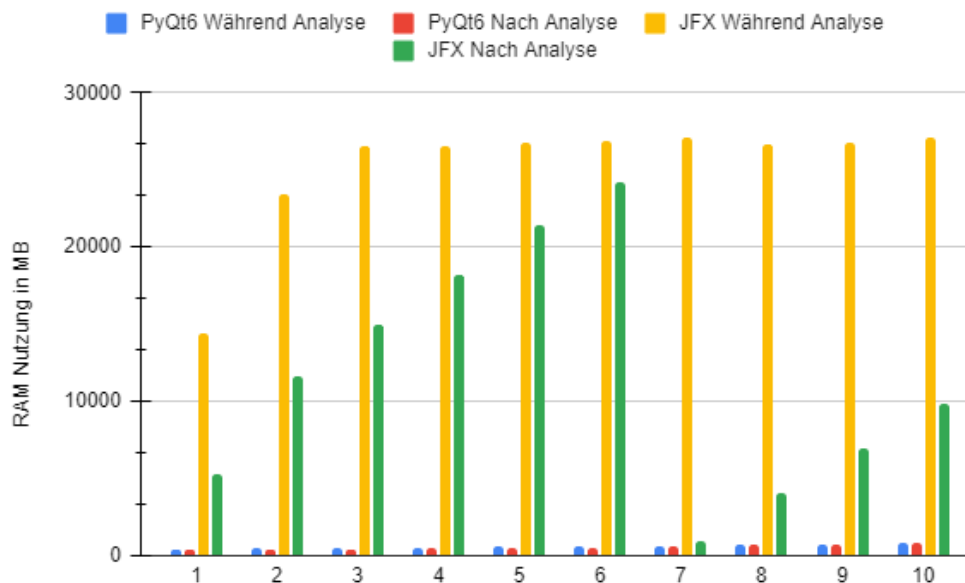


Abbildung 7.2: Vergleich der *RAM*-Nutzung während und nach der Analyse der verschiedenen Anwendungen

In Abbildung 7.2 lässt sich sehr gut erkennen, dass die *JavaFX*-Anwendung eklatant mehr *RAM* benötigt als die *PyQt6*-Anwendung. Wie zuvor kann ich aufgrund meiner Kenntnisse keine Mutmaßungen darüber anstellen.

Zusammenfassend lässt sich sagen, dass die *PyQt6*-Anwendung nicht nur schneller eine Videoeingabe verarbeitet als sein *JavaFX*-Kontrahent, sondern dabei auch deutlich weniger Ressourcen benötigt. Es wirkt so, als ob sich *PyQt6* im Rahmen dieser Aufgabe deutlich besser eignet als *JavaFX*. Zu *UWP* kann ich an dieser Stelle keine Aussage treffen.

Fazit und Ausblick

8.1	Fazit	133
8.2	Ausblick	134

8.1 Fazit

Man sagt "Alle Wege führen nach Rom" und diese Arbeit hat gezeigt, dass dieses Sprichwort auch auf die Anwendungsentwicklung zutrifft. Auf dem Markt gibt es unzählige Werkzeuge mit welchen man eine Anwendung entwickeln kann. Selbst wenn man in der Lage ist den Aufgabenumfang der Anwendung genau zu definieren und genau festlegen zu können, unter welchen technischen Rahmenbedingungen die Anwendung funktionieren muss, bleibt noch immer ein ganzer Stapel an Werkzeugen übrig. Berücksichtigen muss man auch, welche eigenen Kenntnisse vorhanden sind. Doch nur, weil man eine Programmiersprache besser beherrscht als andere, ist diese noch lange nicht am besten dafür geeignet, die vor einem liegende Aufgabe zu bewältigen.

Diese Arbeit hat gezeigt, dass Werkzeuge für die gängigsten Programmiersprachen grundsätzlich gut dokumentiert und erlernbar sind. Es lohnt sich dabei auch die eigene Komfortzone zu verlassen und ein Werkzeug für eine Programmiersprache zu benutzen, in welchem man weniger geübt ist.

Bei der Einbindung von neuronalen Netzen hat sich gezeigt, dass *Python* aus Last- und Performanzperspektive wohl am besten geeignet ist im Rahmen der untersuchten Werkzeuge. Jedoch kann zu *UWP* und *C++* keine Aussage getroffen werden.

8.2 Ausblick

Auch wenn die Arbeit gezeigt hat, dass *Python* sowie *PyQt6* für die GUI-Entwicklung besser geeignet sind als *Java* und *JavaFX*, heißt das noch lange nicht, dass *Python/PyQt6* auch generell die beste Wahl darstellen, um eine *Windows*-Desktopanwendung zu entwickeln. Hier sollte gerade die Programmiersprache *C++* untersucht werden. Da das *Qt*-Framework auf *C++* aufbaut, könnte man die bereits vorhandene mit *Qt* entwickelte GUI zu nutzen und sie an ein *C++*- und nicht an ein *Python-Backend* zu knüpfen. Generell kann man verschiedene Teile der hier entwickelten Anwendungen für Tests in anderen Anwendungen verwenden, da die Analyse einer Videoeingabe mithilfe eines neuronalen Netzes innerhalb der Programmiersprache stattfindet und nur für das Anzeigen der Daten auf der GUI auch diese angebunden werden muss. Unterschiedliche *guis* können natürlich auch verschiedene Performanzzeiten haben.

Ebenso wurde nur teilweise der Performanzunterschied zwischen verschiedenen Konvertierungstypen eines neuronalen Netzes gemessen. Andere Konvertierungstypen könnten andere Last- und Performanzwerte vorweisen und gegebenenfalls eine bessere Wahl als *ONNX* darstellen.

Außerdem könnte sich ein Blick in Richtung *R* und *Julia* lohnen, da diese hier nicht behandelt wurden. Allerdings habe ich bei den beiden genannten keinerlei Kenntnisse, sodass ich nicht weiter über deren Eignung mutmaßen kann.

Gleiches gilt für andere Programmiersprachen um eine *gui* zu bauen. Hier wurden nur Möglichkeiten untersucht, ein neuronales Netz ohne Konvertierung in eine andere Programmiersprache zu nutzen. Als Beispiel könnte eine Mischung aus *HTML* und *C#* auf einem ähnlichen oder gar besseren Niveau arbeiten als *Python/PyQt6*.

Am Ende können die Ergebnisse dieser Arbeit genutzt werden, um darauf weitere Forschungen für die Eignung verschiedener Programmiersprachen und Werkzeuge zur Integration von neuronalen Netzen in eine Desktopanwendung zur medizinischen Bildanalyse durchzuführen, damit jeder sein eigenes "Rom" finden kann.